Set-Coded Turing Machines in Isabelle/ZF

Tang Ziyi

October 14, 2026

Abstract

We construct an explicit closed set-theoretic sentence for the halting of a finite Turing machine on a finite input. For every transitive ZF set model, satisfaction of the sentence is equivalent to external execution reaching the final state. The construction uses set-coded machines, finite execution witnesses, and formulas naming finite objects. A primitive-recursive function constructs the sentence's natural-number code from independent machine-code and unary-input arguments. The library connects concrete program proofs to the existing Isabelle/ZF model interfaces. Observational tape equality and worked clients support reuse without unfolding the internal formulas.

Contents

1 Result **1**

2 Proof **1**
2.1 Finite witnesses 1
2.2 Closed sentences and codes 2

3 Use **3**
3.1 Machine semantics 3
3.2 Representation independence 3
3.3 A complete client 3

4 Sources and scope **4**

5 Set-Coded Turing Machines **4**
5.1 Machine Representation 4
5.2 Tape Semantics 6
5.3 One-Step Semantics 7
5.4 Finite Computations 9
5.5 Numeral Inputs 10
5.6 Halting 11

6 Observable Tape Contents **12**
6.1 Input Representation and Halting 14
6.2 Tape Span and Trailing Blanks 14

7 Using the Machine Interface **15**
7.1 Changing the finite representation 16

8 Machine Coding **16**
8.1 Natural Pairing 17
8.2 Natural Lists 19
8.3 Instruction Streams 21
8.4 Machine Numbers 22

9 Operations on Machine and List Codes **23**
9.1 Instruction Code Normalization 27

10 Primitive-Recursive Coding **28**
10.1 Composition and Projection 28
10.2 Arithmetic 29
10.3 Conditionals 30
10.4 Cantor Pairing 32
10.5 Cantor Projections 32
10.6 Coded Natural Lists 34

11 Primitive-Recursive Instruction Normalization **36**

12 Finite Witnesses in Transitive Models **38**
12.1 Finite object closure 38
12.2 Transitive model interpretations 39
12.3 Machine objects in a set model 39

13 Natural Membership in Transitive Models **39**

14 Internal Halting Formula **40**
14.1 Names for natural numbers 40
14.2 Model and tape updates 41
14.3 Iterated tails 43
14.4 Fetching an instruction 45
14.5 One step 46
14.6 Run formulas 47
14.7 Run correspondence 49
14.8 Halting satisfaction 49
14.9 Arities 50

15 Closed Halting Sentences **50**
15.1 Names for finite lists 50
15.2 Closed halting sentences 52
15.3 Model-theoretic consequences 53

16 Numerical Coding of Formulas **53**
16.1 Constructor codes 53
16.2 Injectivity 54

17 Primitive-Recursive Formula Constructors **54**
17.1 Tagged constructors 54
17.2 Logical connectives 55

18 Primitive-Recursive Names for Numerals **56**
18.1 Natural number names 57
18.2 Finite list names 57
18.3 Unary input names 58

19 Primitive-Recursive Machine Names **58**
19.1 Instruction names 59
19.2 Decoded list indices 59
19.3 Bounded reverse traversal 60
19.4 Complete machine names 62

20 Effective Halting Sentences **62**
20.1 Closed halting sentence code 63
20.2 Self-input instance 63

21 Using the Halting Formula Interface **64**
21.1 From execution to satisfaction 64
21.2 From a sentence code to satisfaction 64

1 Result

For a well-formed machine P and finite symbol input x, let $H(P, x)$ denote the constructed formula *halt_sentence*(P, x). It is a sentence, with no free variables. For every transitive ZF set model M,

$$M \models H(P, x) \quad \Longleftrightarrow \quad \text{halts_on}(P, x).$$

Here halting means that a finite number of external machine steps reaches the final state. The sentence is constructed from P and x alone and is the same for every model. A proof of a particular program's halting or nonhalting therefore determines the truth of its sentence in each such model. The result is conditional on the model premise and asserts no model existence.

The public contract consists of three theorems.

Formula membership	`halt_sentence_type`
No free variables	`arity_halt_sentence`
Satisfaction	`sats_halt_sentence_iff`

Their object premises are $P \in$ *tm_machine* and $x \in$ *list*(*symbol*). The predicate `transitive_zf_model` requires a transitive set satisfying the internalised ZF axiom set from `Internal_ZFC_Axioms`. This is a property of the model under consideration, distinct from the ambient Isabelle/ZF logic.

There is also an effective syntax interface. For natural e, n, let $P_e =$ *decode_machine*(e) and $x_n =$ *numeral_input*(n), the list of n ones. The certified primitive-recursive constructor F satisfies

$$F(e, n) = \ulcorner H(P_e, x_n) \urcorner.$$

Thus a program and its input determine both a closed sentence and its code. Section 20 proves membership in the bundled `prim_rec` class and the application equation. This is a certificate for numerical sentence construction. It does not supply a compiler implementing arbitrary primitive-recursive functions in the machine model.

2 Proof

2.1 Finite witnesses

A halting execution has a finite trace. If P halts on x at time n, the canonical witness is

$$r = \lambda k \in n + 1.\, \text{steps}(P, \text{initial_config}(x), k).$$

Each configuration is built from natural numbers and finite lists. The model contains these objects and their ordered pairs. Finite induction then places the entire trace in the model. The closure theorem `tm_finite_lambda_`

`closed` requires membership of each value; it does not require an external function to have been supplied internally. `transitive_zf_contains_trace` applies this argument to runs.

The explicit formula `halt_fm` binds a natural time and a witness set. It constrains the witness's value at time zero, each successive transition, and the final state. The canonical trace satisfies these conditions. Conversely, suppose an internal witness satisfies them. Induction identifies each constrained value with the corresponding external configuration. The one-step satisfaction theorem provides the induction step. At the final time, the external machine must therefore be in state zero.

These directions have different obligations. The external execution supplies a finite function that belongs to the model. An arbitrary internal witness need only satisfy the stated application constraints; its entire set need not be a finite function. No interpretation of an unbounded execution inside the model is assumed.

The shared lemmas `trans_run_steps` and `trace_trans`, in the locale `M_ZF1_trans`, give reconstruction and canonical transitions for arbitrary initial configurations and outer formula environments. They isolate the execution argument from a surrounding formula's variable layout. The halting theorem and downstream output predicates use the same lemmas. Section 14 contains the explicit formula and the proofs.

2.2 Closed sentences and codes

The formula `halt_fm` has environment $[P, x]$. Formulas naming natural numbers, instructions, and finite lists define P and x uniquely. Binding these names yields $H(P, x)$ and discharges both free variables. This construction uses the formula datatype of `ZF-Constructible.Formula` [6].

`Formula_Coding` assigns tags and paired natural-number codes to the four formula constructors. Structural induction proves injectivity. The primitive-recursive name constructors mirror the syntactic construction. A bounded reverse traversal builds the name of a decoded machine, whose list length is at most its code. The resulting application equation identifies $F(e, n)$ with the code of $H(P_e, x_n)$. Formula-code injectivity then identifies any well-formed formula bearing that code, so no separate formula decoder is needed for this contract.

The implementation of F is `pr_machine_input_sentence_code`, applied to $[e, n]$. The associated theorem, `sats_machine_input_sentence_code_iff`, derives satisfaction directly from the constructed code. Setting $n = e$ yields the unary constructor `pr_halt_sentence_code`. These certificates use the Isabelle/ZF primitive-recursive function class [7].

3 Use

3.1 Machine semantics

A machine is a finite list of instructions over the symbols 0 and 1. Each instruction contains an action and a next state. The actions write 0, write 1, move left, move right, or leave the tape unchanged. State 1 is initial and state 0 is final. The two entries for a positive state q occur at indices $2(q-1)$ and $2(q-1)+1$. Missing entries enter the final state without changing the tape. The final state is absorbing.

A configuration contains a state and two finite symbol lists. The left list starts immediately left of the head, while the right list starts at the scanned cell. Unrepresented cells are blank. `tm_step` and `steps` specify one step and finite iteration. The predicate `halts_on` includes machine and input validity. Theorem `halts_on_iff_finite_run` supplies the equivalent finite-function witness form. For nonhalting proofs, `not_halts_on_invariant` accepts an initial, step-closed predicate that excludes the final state.

3.2 Representation independence

The lists [1] and $[1, 0, 0]$ are distinct stored objects with equal observed contents. The relations `half_tape_eq`, `tape_eq`, and `config_eq` compare observations at fixed positions relative to the head. They ignore trailing stored blanks and preserve execution and halting. They do not identify head shifts. Literal storage length remains a separate property.

The theorem `sats_halt_sentence_half_tape_eq_iff` transfers internal halting truth between observationally equal inputs. The sentences may have different syntax and codes because they name different lists. Their satisfaction agrees.

3.3 A complete client

`Turing_Examples` defines a two-instruction machine that scans a unary input and writes one at the first blank. Its loop invariant gives the final configuration after $n+1$ steps. The head remains on the newly written cell. The proof derives halting and transfers the result to an input with an extra stored blank.

`Turing_Halting_Examples` continues this same example. The theorem `scan_append_sentence` obtains internal truth from the operational halting result. The theorem `scan_append_sentence_code` obtains truth of any formula bearing the constructed code. Neither proof unfolds the halting formula, the finite-object names, or the numerical constructor. The operational example can be imported without model theory or coding.

4 Sources and scope

The instruction-list and two-list tape design follows the Isabelle/HOL development of Xu, Zhang, and Urban [1] and its AFP entry *Universal Turing Machine* [3]. The representation uses the unshifted `tprog0` convention, with natural-number actions and symbols and no program-offset parameter. Here machines and finite runs are sets whose membership in a model can be proved in the same logic as the satisfaction statements.

Closure and satisfaction use the AFP model infrastructure [4]. The `Independence_CH` entry supplies the locale and full-ZF interpretation in `ZF_Trans_Interpretations` [2]. The proofs apply in the weaker locale `M_ZF1_trans`, which assumes transitivity, Zermelo set theory, and four specified replacement instances. The full-ZF results interpret this locale. No minimality of these assumptions is claimed. No CH independence result or choice assumption is used by the halting interface.

The underlying absoluteness method is classical [5, Chapter 12]. The contribution is the explicit formula, its effective closed instances, and their verified connection to concrete machine execution. The semantic interface accepts all finite symbol inputs; the numerical constructor uses unary inputs. Machine decoding normalises invalid instruction fields and is total on natural numbers. Encoding and then decoding a valid machine recovers it, while arbitrary machine codes need not be unique.

5 Set-Coded Turing Machines

theory *Turing-Machine*
  imports *ZF*
begin

5.1 Machine Representation

A machine is a finite instruction list. An instruction contains an action number and a next-state number. Actions 0 and 1 write symbols, 2 and 3 move the head, and 4 leaves the tape unchanged. State 1 is initial and state 0 is final. The named actions, states, and symbols are input abbreviations for these natural numbers. Printed propositions use numbers so that the same set, such as 0, is not given an action name when it occurs as a state or symbol.

definition *symbol* :: *i* **where**
  $symbol \equiv 2$

lemma *symbol-nat*: $a \in symbol \Longrightarrow a \in nat$
  ⟨*proof*⟩

definition *action* :: *i* **where**

$action \equiv succ(succ(succ(2)))$

lemma *action-in-nat* [*TC*]:
$action \in nat$
⟨*proof*⟩

lemma *action-subset-nat*:
$action \subseteq nat$
⟨*proof*⟩

abbreviation (*input*) *blank-symbol* :: *i* **where**
$blank\text{-}symbol \equiv 0$

abbreviation (*input*) *one-symbol* :: *i* **where**
$one\text{-}symbol \equiv 1$

abbreviation (*input*) *final-state* :: *i* **where**
$final\text{-}state \equiv 0$

abbreviation (*input*) *initial-state* :: *i* **where**
$initial\text{-}state \equiv 1$

abbreviation (*input*) *write-blank* :: *i* **where**
$write\text{-}blank \equiv 0$

abbreviation (*input*) *write-one* :: *i* **where**
$write\text{-}one \equiv 1$

abbreviation (*input*) *move-left* :: *i* **where**
$move\text{-}left \equiv 2$

abbreviation (*input*) *move-right* :: *i* **where**
$move\text{-}right \equiv succ(2)$

abbreviation (*input*) *nop* :: *i* **where**
$nop \equiv succ(succ(2))$

definition *tm-tape* :: *i* **where**
$tm\text{-}tape \equiv list(symbol) \times list(symbol)$

definition *instruction* :: *i* **where**
$instruction \equiv action \times nat$

definition *tm-machine* :: *i* **where**
$tm\text{-}machine \equiv list(instruction)$

definition *configuration* :: *i* **where**
$configuration \equiv nat \times tm\text{-}tape$

lemma *symbol-type* [*simp*,*TC*]:
shows *blank-symbol* ∈ *symbol*
and *one-symbol* ∈ *symbol*
⟨*proof*⟩

lemma *action-type* [*simp*,*TC*]:
shows *write-blank* ∈ *action*
and *write-one* ∈ *action*
and *move-left* ∈ *action*
and *move-right* ∈ *action*
and *nop* ∈ *action*
⟨*proof*⟩

lemma *tapeI* [*intro*]:
assumes *left*: *l* ∈ *list*(*symbol*)
and *right*: *r* ∈ *list*(*symbol*)
shows ⟨*l*,*r*⟩ ∈ *tm-tape*
⟨*proof*⟩

lemma *tapeD*:
assumes *tm-tape*: *t* ∈ *tm-tape*
shows *fst*(*t*) ∈ *list*(*symbol*)
and *snd*(*t*) ∈ *list*(*symbol*)
⟨*proof*⟩

5.2 Tape Semantics

The left list begins with the cell immediately left of the head; the right list begins with the scanned cell. Cells beyond either finite list are blank. This representation uses the Isabelle/ZF equations hd([]) = 0 and nth(n,[]) = 0, with blank symbol 0.

definition *scan* :: *i* ⇒ *i* **where**
scan(*t*) ≡ *hd*(*snd*(*t*))

lemma *hd-symbol-type*:
assumes *symbols*: *xs* ∈ *list*(*symbol*)
shows *hd*(*xs*) ∈ *symbol*
⟨*proof*⟩

lemma *scan-type* [*TC*]:
assumes *tm-tape*: *t* ∈ *tm-tape*
shows *scan*(*t*) ∈ *symbol*
⟨*proof*⟩

lemma *scan-pair* [*simp*]:
scan(⟨*l*,*r*⟩) = *hd*(*r*)
⟨*proof*⟩

definition *tm-update* :: [*i*,*i*] ⇒ *i* **where**

```
  tm-update(a,t) ≡
    if a = write-blank then
      ⟨fst(t),Cons(blank-symbol,tl(snd(t)))⟩
    else if a = write-one then
      ⟨fst(t),Cons(one-symbol,tl(snd(t)))⟩
    else if a = move-left then
      ⟨tl(fst(t)),Cons(hd(fst(t)),snd(t))⟩
    else if a = move-right then
      ⟨Cons(hd(snd(t)),fst(t)),tl(snd(t))⟩
    else t

lemma tm-update-type [TC]:
  assumes tm-tape: t ∈ tm-tape
  shows tm-update(a,t) ∈ tm-tape
⟨proof⟩

lemma tm-update-write-blank [simp]:
  tm-update(write-blank,t) =
    ⟨fst(t),Cons(blank-symbol,tl(snd(t)))⟩
  ⟨proof⟩

lemma tm-update-write-one [simp]:
  tm-update(write-one,t) =
    ⟨fst(t),Cons(one-symbol,tl(snd(t)))⟩
  ⟨proof⟩

lemma tm-update-move-left [simp]:
  tm-update(move-left,t) =
    ⟨tl(fst(t)),Cons(hd(fst(t)),snd(t))⟩
  ⟨proof⟩

lemma tm-update-move-right [simp]:
  tm-update(move-right,t) =
    ⟨Cons(hd(snd(t)),fst(t)),tl(snd(t))⟩
  ⟨proof⟩

lemma tm-update-nop [simp]:
  tm-update(nop,t) = t
  ⟨proof⟩
```

5.3 One-Step Semantics

For a nonfinal state q and scanned symbol b, the instruction slot is 2 * pred(q) + b. A missing slot enters the final state without changing the tape. The final state is absorbing.

```
definition slot :: [i,i] ⇒ i where
  slot(q,b) ≡ 2 #* pred(q) #+ b

lemma slot-type [TC]:
```

assumes *state*: $q \in nat$
and *scanned*: $b \in symbol$
shows $slot(q,b) \in nat$
⟨*proof*⟩

lemma *slot-succ* [*simp*]:
assumes *state*: $q \in nat$
shows $slot(succ(q),b) = 2 \#* q \#+ b$
⟨*proof*⟩

definition *fetch* :: $[i,i,i] \Rightarrow i$ **where**
$fetch(M,q,b) \equiv$
if $q =$ *final-state then* ⟨*nop*,*final-state*⟩
else if $slot(q,b) < length(M)$ *then*
$nth(slot(q,b),M)$
else ⟨*nop*,*final-state*⟩

lemma *fetch-type* [*TC*]:
assumes *tm-machine*: $M \in$ *tm-machine*
shows $fetch(M,q,b) \in$ *instruction*
⟨*proof*⟩

lemma *fetch-final* [*simp*]:
$fetch(M,$*final-state*$,b) = ⟨nop,$*final-state*⟩
⟨*proof*⟩

lemma *fetch-in-range* [*simp*]:
assumes *nonfinal*: $q \neq$ *final-state*
and *in-range*: $slot(q,b) < length(M)$
shows $fetch(M,q,b) = nth(slot(q,b),M)$
⟨*proof*⟩

lemma *fetch-out-of-range* [*simp*]:
assumes *out-of-range*: $\neg\ slot(q,b) < length(M)$
shows $fetch(M,q,b) = ⟨nop,$*final-state*⟩
⟨*proof*⟩

definition *tm-step* :: $[i,i] \Rightarrow i$ **where**
tm-step$(M,c) \equiv$
let ins = $fetch(M,fst(c),scan(snd(c)))$
in $\langle snd(ins),$*tm-update*$(fst(ins),snd(c))\rangle$

lemma *step-type* [*TC*]:
assumes *tm-machine*: $M \in$ *tm-machine*
and *configuration*: $c \in$ *configuration*
shows *tm-step*$(M,c) \in$ *configuration*
⟨*proof*⟩

lemma *step-pair* [*simp*]:

tm-step(M,$\langle q,t\rangle$) =
$\langle$*snd*(*fetch*(M,q,*scan*(t))),
tm-update(*fst*(*fetch*(M,q,*scan*(t))),t)$\rangle$
⟨*proof*⟩

lemma *step-final* [*simp*]:
tm-step(M,$\langle$*final-state*,$t\rangle$) = $\langle$*final-state*,$t\rangle$
⟨*proof*⟩

lemma *step-final-configuration* [*simp*]:
assumes *configuration*: $c \in$ *configuration*
and *final*: *fst*(c) = *final-state*
shows *tm-step*(M,c) = c
⟨*proof*⟩

5.4 Finite Computations

A finite run is a set-theoretic function on succ(n). The equivalence theorem identifies such a run with n iterations of the step function. This witness form supports internal descriptions of finite computations in set-theoretic models.

definition *steps* :: [i,i,i] ⇒ i **where**
steps(M,c,n) ≡ (λx. *tm-step*(M,x))^n (c)

lemma *steps-0* [*simp*]:
steps(M,c,0) = c
⟨*proof*⟩

lemma *steps-succ* [*simp*]:
steps(M,c,*succ*(n)) = *tm-step*(M,*steps*(M,c,n))
⟨*proof*⟩

lemma *steps-type* [*TC*]:
assumes *tm-machine*: $M \in$ *tm-machine*
and *configuration*: $c \in$ *configuration*
and *time*: $n \in$ *nat*
shows *steps*(M,c,n) ∈ *configuration*
⟨*proof*⟩

lemma *steps-add*:
assumes *m-nat*: $m \in$ *nat*
and *n-nat*: $n \in$ *nat*
shows *steps*(M,c,m #+ n) = *steps*(M,*steps*(M,c,m),n)
⟨*proof*⟩

lemma *steps-invariant*:
assumes *time*: $n \in$ *nat* **and** *start*: $I(c)$
and *closed*: $\bigwedge d$. $I(d) \Longrightarrow I$(*tm-step*(P,d))
shows I(*steps*(P,c,n))

⟨*proof*⟩

lemma *steps-final* [*simp*]:
assumes *time*: $n \in nat$
shows *steps*(M,⟨*final-state*,t⟩,n) =
⟨*final-state*,t⟩
⟨*proof*⟩

lemma *steps-final-absorb*:
assumes *machine-M*: $M \in$ *tm-machine*
and *configuration*: $c \in$ *configuration*
and *earlier*: $m \in nat$
and *later*: $n \in nat$
and *ordered*: $m \leq n$
and *final*: *fst*(*steps*(M,c,m)) = *final-state*
shows *steps*(M,c,n) = *steps*(M,c,m)
⟨*proof*⟩

lemma *steps-final-mono*:
assumes *machine-M*: $M \in$ *tm-machine*
and *configuration*: $c \in$ *configuration*
and *earlier*: $m \in nat$
and *later*: $n \in nat$
and *ordered*: $m \leq n$
and *final*: *fst*(*steps*(M,c,m)) = *final-state*
shows *fst*(*steps*(M,c,n)) = *final-state*
⟨*proof*⟩

definition *finite-run* :: [i,i,i] ⇒ o **where**
finite-run(M,n,r) ≡
relation(r) ∧
function(r) ∧
domain(r) = *succ*(n) ∧
(∀ k∈n. r‘*succ*(k) = *tm-step*(M,r‘k))

lemma *finite-run-iff-steps*:
assumes *time*: $n \in nat$
shows *finite-run*(M,n,r) ∧ r‘0 = c ⟷
r = (λk∈*succ*(n). *steps*(M,c,k))
⟨*proof*⟩

5.5 Numeral Inputs

definition *numeral-input* :: $i \Rightarrow i$ **where**
numeral-input(n) ≡ *repeat*(*one-symbol*,n)

lemma *numeral-input-type* [*TC*]:
assumes *natural*: $n \in nat$
shows *numeral-input*(n) ∈ *list*(*symbol*)

⟨*proof*⟩

lemma *length-numeral-input* [*simp*]:
assumes *natural*: $n \in nat$
shows *length*(*numeral-input*(n)) = n
⟨*proof*⟩

lemma *numeral-input-injective*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
and *equal*: *numeral-input*(m) = *numeral-input*(n)
shows $m = n$
⟨*proof*⟩

5.6 Halting

abbreviation *initial-config* :: $i \Rightarrow i$ **where**
initial-config(x) ≡ ⟨*initial-state*,⟨[],x⟩⟩

lemma *initial-config-type* [TC]:
assumes *input*: $x \in list(symbol)$
shows *initial-config*(x) ∈ *configuration*
⟨*proof*⟩

definition *halts-on* :: [i,i] ⇒ o **where**
halts-on(M,x) ≡
$M \in$ *tm-machine* ∧
$x \in list(symbol)$ ∧
(∃ n∈*nat*.
fst(*steps*(M,*initial-config*(x),n)) = *final-state*)

lemma *halts-onI*:
assumes *machine*: $P \in$ *tm-machine* **and** *input*: $x \in list(symbol)$
and *time*: $n \in nat$
and *final*: *fst*(*steps*(P,*initial-config*(x),n)) = *final-state*
shows *halts-on*(P,x)
⟨*proof*⟩

A nonhalting proof may give a step-closed invariant that excludes the final state. Only reachable configurations need to satisfy the invariant. The rule requires neither a running-time bound nor a complete transition table.

theorem *not-halts-on-invariant*:
assumes *start*: I(*initial-config*(x))
and *closed*: $\bigwedge c.\ I(c) \Longrightarrow I$(*tm-step*($P$,$c$))
and *nonfinal*: $\bigwedge c.\ I(c) \Longrightarrow$ *fst*(c) ≠ *final-state*
shows ¬ *halts-on*(P,x)
⟨*proof*⟩

theorem *halts-on-iff-finite-run*:

halts-on(*M*,*x*) ⟷
M ∈ *tm-machine* ∧
x ∈ *list*(*symbol*) ∧
(∃ *n*∈*nat*. ∃ *r*.
finite-run(*M*,*n*,*r*) ∧
r‘0 = *initial-config*(*x*) ∧
fst(*r*‘*n*) = *final-state*)
⟨*proof*⟩

abbreviation *halts-blank* :: *i* ⇒ *o* **where**
halts-blank(*M*) ≡ *halts-on*(*M*,[])

end

6 Observable Tape Contents

theory *Turing-Tape*
imports *Turing-Machine*
begin

A finite list represents an infinite blank-ended half-tape. The Isabelle/ZF function nth returns 0 beyond the end of the list, matching the blank symbol. Equality of represented contents therefore ignores trailing blanks. The relation *half-tape-eq* compares observations only. Its uses as a tape interface state list membership separately. The relations *tape-eq* and *config-eq* include the validity of both arguments.

definition *half-tape-eq* :: [*i*,*i*] ⇒ *o* **where**
half-tape-eq(*xs*,*ys*) ≡ ∀ *n*∈*nat*. *nth*(*n*,*xs*) = *nth*(*n*,*ys*)

lemma *half-tape-eq-refl* [*simp*]: *half-tape-eq*(*xs*,*xs*)
⟨*proof*⟩

lemma *half-tape-eq-sym*: *half-tape-eq*(*xs*,*ys*) ⟹ *half-tape-eq*(*ys*,*xs*)
⟨*proof*⟩

lemma *half-tape-eq-trans*:
⟦*half-tape-eq*(*xs*,*ys*); *half-tape-eq*(*ys*,*zs*)⟧ ⟹ *half-tape-eq*(*xs*,*zs*)
⟨*proof*⟩

lemma *nth-zero-hd*:
xs ∈ *list*(*A*) ⟹ *nth*(*0*,*xs*) = *hd*(*xs*)
⟨*proof*⟩

lemma *nth-succ-tl*:
⟦*xs* ∈ *list*(*A*); *n* ∈ *nat*⟧ ⟹ *nth*(*succ*(*n*),*xs*) = *nth*(*n*,*tl*(*xs*))
⟨*proof*⟩

lemma *half-tape-eq-hd*:

⟦*half-tape-eq*(*xs*,*ys*); *xs* ∈ *list*(*A*); *ys* ∈ *list*(*B*)⟧ ⟹ *hd*(*xs*) = *hd*(*ys*)
⟨*proof*⟩

lemma *half-tape-eq-tl*:
assumes *eq*: *half-tape-eq*(*xs*,*ys*)
and *xs*: *xs* ∈ *list*(*A*) **and** *ys*: *ys* ∈ *list*(*B*)
shows *half-tape-eq*(*tl*(*xs*),*tl*(*ys*))
⟨*proof*⟩

lemma *half-tape-eq-Cons*:
assumes *eq*: *half-tape-eq*(*xs*,*ys*)
shows *half-tape-eq*(*Cons*(*a*,*xs*),*Cons*(*a*,*ys*))
⟨*proof*⟩

lemma *half-tape-eq-blank* [*simp*]: *half-tape-eq*([*blank-symbol*],[])
⟨*proof*⟩

definition *tape-eq* :: [*i*,*i*] ⇒ *o* **where**
tape-eq(*t*,*u*) ≡ *t* ∈ *tm-tape* ∧ *u* ∈ *tm-tape* ∧
half-tape-eq(*fst*(*t*),*fst*(*u*)) ∧ *half-tape-eq*(*snd*(*t*),*snd*(*u*))

lemma *tape-eq-refl*: *t* ∈ *tm-tape* ⟹ *tape-eq*(*t*,*t*)
⟨*proof*⟩

lemma *tape-eq-sym*: *tape-eq*(*t*,*u*) ⟹ *tape-eq*(*u*,*t*)
⟨*proof*⟩

lemma *tape-eq-trans*:
⟦*tape-eq*(*t*,*u*); *tape-eq*(*u*,*v*)⟧ ⟹ *tape-eq*(*t*,*v*)
⟨*proof*⟩

lemma *tape-eq-scan*: *tape-eq*(*t*,*u*) ⟹ *scan*(*t*) = *scan*(*u*)
⟨*proof*⟩

lemma *tape-eq-update*:
assumes *eq*: *tape-eq*(*t*,*u*)
shows *tape-eq*(*tm-update*(*a*,*t*),*tm-update*(*a*,*u*))
⟨*proof*⟩

definition *config-eq* :: [*i*,*i*] ⇒ *o* **where**
config-eq(*c*,*d*) ≡ *c* ∈ *configuration* ∧ *d* ∈ *configuration* ∧
fst(*c*) = *fst*(*d*) ∧ *tape-eq*(*snd*(*c*),*snd*(*d*))

lemma *config-eq-refl*: *c* ∈ *configuration* ⟹ *config-eq*(*c*,*c*)
⟨*proof*⟩

lemma *config-eq-sym*: *config-eq*(*c*,*d*) ⟹ *config-eq*(*d*,*c*)
⟨*proof*⟩

lemma *config-eq-trans*:
⟦*config-eq*(*c*,*d*); *config-eq*(*d*,*e*)⟧ ⟹ *config-eq*(*c*,*e*)
⟨*proof*⟩

lemma *config-eq-pair*:
⟦*q* ∈ *nat*; *tape-eq*(*t*,*u*)⟧ ⟹ *config-eq*(⟨*q*,*t*⟩,⟨*q*,*u*⟩)
⟨*proof*⟩

lemma *config-eq-observe*:
config-eq(*c*,*d*) ⟹ *fst*(*c*) = *fst*(*d*)
config-eq(*c*,*d*) ⟹ *scan*(*snd*(*c*)) = *scan*(*snd*(*d*))
⟨*proof*⟩

lemma *step-config-eq*:
assumes *M*: *M* ∈ *tm-machine* **and** *eq*: *config-eq*(*c*,*d*)
shows *config-eq*(*tm-step*(*M*,*c*),*tm-step*(*M*,*d*))
⟨*proof*⟩

theorem *steps-config-eq*:
assumes *M*: *M* ∈ *tm-machine* **and** *eq*: *config-eq*(*c*,*d*) **and** *n*: *n* ∈ *nat*
shows *config-eq*(*steps*(*M*,*c*,*n*),*steps*(*M*,*d*,*n*))
⟨*proof*⟩

6.1 Input Representation and Halting

lemma *initial-config-eq*:
assumes *x*: *x* ∈ *list*(*symbol*) **and** *y*: *y* ∈ *list*(*symbol*)
and *eq*: *half-tape-eq*(*x*,*y*)
shows *config-eq*(*initial-config*(*x*),*initial-config*(*y*))
⟨*proof*⟩

theorem *halts-on-half-tape-eq-iff*:
assumes *P*: *P* ∈ *tm-machine*
and *x*: *x* ∈ *list*(*symbol*) **and** *y*: *y* ∈ *list*(*symbol*)
and *eq*: *half-tape-eq*(*x*,*y*)
shows *halts-on*(*P*,*x*) ⟷ *halts-on*(*P*,*y*)
⟨*proof*⟩

6.2 Tape Span and Trailing Blanks

definition *tape-span* :: *i* ⇒ *i* **where**
tape-span(*t*) ≡ *length*(*fst*(*t*)) #+ *length*(*snd*(*t*))

lemma *tm-update-span-mono*:
assumes *t*: *t* ∈ *tm-tape*
shows *tape-span*(*t*) ≤ *tape-span*(*tm-update*(*a*,*t*))
⟨*proof*⟩

lemma *step-span-mono*:
c ∈ *configuration* ⟹ *tape-span*(*snd*(*c*)) ≤ *tape-span*(*snd*(*tm-step*(*M*,*c*)))

⟨*proof*⟩

lemma *steps-span-mono*:
assumes *M*: $M \in$ *tm-machine* **and** *c*: $c \in$ *configuration* **and** *n*: $n \in$ *nat*
shows *tape-span*(*snd*(*c*)) $\leq$ *tape-span*(*snd*(*steps*(*M*,*c*,*n*)))
⟨*proof*⟩

lemma *half-tape-eq-blank-repeat*:
assumes *n*: $n \in$ *nat*
shows *half-tape-eq*(*repeat*(*blank-symbol*,*n*),[])
⟨*proof*⟩

lemma *half-tape-eq-append*:
⟦*xs* ∈ *list*(*symbol*); *half-tape-eq*(*ys*,*zs*)⟧ ⟹ *half-tape-eq*(*xs* @ *ys*,*xs* @ *zs*)
⟨*proof*⟩

lemma *half-tape-eq-trailing-blank*:
xs ∈ *list*(*symbol*) ⟹ *half-tape-eq*(*xs* @ [*blank-symbol*],*xs*)
⟨*proof*⟩

end

7 Using the Machine Interface

theory *Turing-Examples*
imports *Turing-Tape*
begin

The machine scans right across a unary input, writes one at the first blank cell, and halts there. Its two instructions belong to state 1. The first is selected on blank, the second on one. The result theorem specifies the complete final configuration, including the head position. It does not impose a convention for returning the head to the input origin.

definition *scan-append-machine* :: *i* **where**
scan-append-machine ≡
[⟨*write-one*,*final-state*⟩,⟨*move-right*,*initial-state*⟩]

lemma *scan-append-machine-type* [*TC*]: *scan-append-machine* ∈ *tm-machine*
⟨*proof*⟩

lemma *scan-append-fetch-blank* [*simp*]:
fetch(*scan-append-machine*,*initial-state*,*blank-symbol*) =
⟨*write-one*,*final-state*⟩
⟨*proof*⟩

lemma *scan-append-fetch-one* [*simp*]:

fetch(*scan-append-machine*,*initial-state*,*one-symbol*) =
⟨*move-right*,*initial-state*⟩
⟨*proof*⟩

The loop invariant allows an arbitrary stored left half-tape. Each step transfers one input symbol to that half. Generalising the left half before induction makes the invariant applicable after the first step.

lemma *scan-append-loop*:
assumes *n*: $n \in nat$ **and** *left*: $l \in list(symbol)$
shows *steps*(*scan-append-machine*,
⟨*initial-state*,⟨*l*,*numeral-input*(*n*)⟩⟩,*n*) =
⟨*initial-state*,⟨*numeral-input*(*n*) @ *l*,[]⟩⟩
⟨*proof*⟩

theorem *scan-append-result*:
assumes $n \in nat$
shows *steps*(*scan-append-machine*,*initial-config*(*numeral-input*(*n*)),*succ*(*n*)) =
⟨*final-state*,⟨*numeral-input*(*n*),[*one-symbol*]⟩⟩
⟨*proof*⟩

corollary *scan-append-halts*:
assumes $n \in nat$
shows *halts-on*(*scan-append-machine*,*numeral-input*(*n*))
⟨*proof*⟩

7.1 Changing the finite representation

Appending a stored blank does not change the observed run. The proof transfers *scan-append-result* through configuration equivalence. It uses the observation interface without repeating the scan induction.

theorem *scan-append-padded-result*:
assumes *n*: $n \in nat$
shows *config-eq*(
steps(*scan-append-machine*,
initial-config(*numeral-input*(*n*) @ [*blank-symbol*]),*succ*(*n*)),
⟨*final-state*,⟨*numeral-input*(*n*),[*one-symbol*]⟩⟩)
⟨*proof*⟩

end

8 Machine Coding

theory *Turing-Coding*
imports *Turing-Machine*
begin

8.1 Natural Pairing

The pairing code enumerates diagonals of nat times nat. The inverse follows the same diagonals by iteration; the two directions are proved inverse on natural numbers.

consts *triangle* :: $i \Rightarrow i$

primrec
  triangle(*0*) = *0*
  triangle(*succ*(*n*)) = *triangle*(*n*) #+ *succ*(*n*)

definition *pair-code* :: [*i*,*i*] ⇒ *i* **where**
  pair-code(*m*,*n*) ≡ *triangle*(*m* #+ *n*) #+ *n*

definition *pair-step* :: *i* ⇒ *i* **where**
  pair-step(*p*) ≡
    if fst(*p*) = *0 then*
      ⟨*succ*(*snd*(*p*)),*0*⟩
    else
      ⟨*pred*(*fst*(*p*)),*succ*(*snd*(*p*))⟩

consts *pair-decode* :: *i* ⇒ *i*

primrec
  pair-decode(*0*) = ⟨*0*,*0*⟩
  pair-decode(*succ*(*n*)) = *pair-step*(*pair-decode*(*n*))

definition *pair-number* :: *i* ⇒ *i* **where**
  pair-number(*p*) ≡ *pair-code*(*fst*(*p*),*snd*(*p*))

lemma *triangle-type* [*TC*]:
  assumes *natural*: *n* ∈ *nat*
  shows *triangle*(*n*) ∈ *nat*
  ⟨*proof*⟩

lemma *pair-code-type* [*TC*]:
  assumes *first*: *m* ∈ *nat*
    and *second*: *n* ∈ *nat*
  shows *pair-code*(*m*,*n*) ∈ *nat*
  ⟨*proof*⟩

lemma *pair-step-type* [*TC*]:
  assumes *pair*: *p* ∈ *nat* × *nat*
  shows *pair-step*(*p*) ∈ *nat* × *nat*
⟨*proof*⟩

lemma *pair-decode-type* [*TC*]:
  assumes *natural*: *n* ∈ *nat*
  shows *pair-decode*(*n*) ∈ *nat* × *nat*

⟨*proof*⟩

lemma *pair-number-type* [*TC*]:
assumes *pair*: $p \in nat \times nat$
shows *pair-number*$(p) \in nat$
⟨*proof*⟩

lemma *pair-number-step*:
assumes *pair*: $p \in nat \times nat$
shows *pair-number*(*pair-step*(p)) = *succ*(*pair-number*(p))
⟨*proof*⟩

lemma *pair-number-decode*:
assumes *natural*: $n \in nat$
shows *pair-number*(*pair-decode*(n)) = n
⟨*proof*⟩

definition *pair-prev* :: $i \Rightarrow i$ **where**
pair-prev$(p) \equiv$
if *snd*(p) = *0* *then*
⟨*0*,*pred*(*fst*(p))⟩
else
⟨*succ*(*fst*(p)),*pred*(*snd*(p))⟩

lemma *pair-prev-type* [*TC*]:
assumes *pair*: $p \in nat \times nat$
shows *pair-prev*$(p) \in nat \times nat$
⟨*proof*⟩

lemma *triangle-eq-zero*:
assumes *natural*: $n \in nat$
shows *triangle*(n) = *0* $\longleftrightarrow$ n = *0*
⟨*proof*⟩

lemma *pair-number-zero*:
assumes *pair*: $p \in nat \times nat$
shows *pair-number*(p) = *0* $\longleftrightarrow$ $p = \langle 0,0 \rangle$
⟨*proof*⟩

lemma *pair-step-prev*:
assumes *pair*: $p \in nat \times nat$
and *nonzero*: $p \neq \langle 0,0 \rangle$
shows *pair-step*(*pair-prev*(p)) = p
⟨*proof*⟩

lemma *pair-decode-number-aux*:
assumes *natural*: $n \in nat$
shows $\forall\, p{\in}nat \times nat.$
pair-number$(p) = n \longrightarrow$ *pair-decode*$(n) = p$

⟨*proof*⟩

lemma *pair-decode-number*:
assumes *pair*: $p \in nat \times nat$
shows *pair-decode*(*pair-number*(p)) = p
⟨*proof*⟩

theorem *pair-decode-code*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
shows *pair-decode*(*pair-code*(m,n)) = ⟨m,n⟩
⟨*proof*⟩

theorem *pair-code-decode*:
assumes *natural*: $n \in nat$
shows *pair-code*(*fst*(*pair-decode*(n)),*snd*(*pair-decode*(n))) = n
⟨*proof*⟩

lemma *pair-decode-iterates*:
assumes *natural*: $n \in nat$
shows *pair-decode*(n) = *pair-step*^n (⟨0,0⟩)
⟨*proof*⟩

theorem *pair-number-bij*:
(λp∈$nat \times nat$. *pair-number*(p))
∈ *bij*($nat \times nat$,nat)
⟨*proof*⟩

lemma *pair-code-second-le*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
shows $n \leq$ *pair-code*(m,n)
⟨*proof*⟩

lemma *pair-decode-second-le*:
assumes *natural*: $n \in nat$
shows *snd*(*pair-decode*(n)) $\leq n$
⟨*proof*⟩

lemma *pair-decode-second-in-succ*:
assumes *natural*: $n \in nat$
shows *snd*(*pair-decode*(n)) ∈ *succ*(n)
⟨*proof*⟩

8.2 Natural Lists

consts *nat-list-encode* :: $i \Rightarrow i$

primrec

nat-list-encode([]) = *0*
nat-list-encode(*Cons*(*a*,*l*)) =
succ(*pair-code*(*a*,*nat-list-encode*(*l*)))

definition *nat-list-decode* :: $i \Rightarrow i$ **where**
nat-list-decode(*n*) ≡
wfrec(*Memrel*(*nat*),*n*,
λ*k f*. *nat-case*([],
λ*m*. *Cons*(*fst*(*pair-decode*(*m*)),*f*‘*snd*(*pair-decode*(*m*))),*k*))

lemmas *nat-list-decode-unfold* =
def-wfrec [*OF nat-list-decode-def wf-Memrel*]

lemma *nat-list-encode-type* [*TC*]:
assumes *list*: *l* ∈ *list*(*nat*)
shows *nat-list-encode*(*l*) ∈ *nat*
⟨*proof*⟩

lemma *nat-list-decode-zero* [*simp*]:
nat-list-decode(*0*) = []
⟨*proof*⟩

lemma *nat-list-decode-succ* [*simp*]:
assumes *natural*: *n* ∈ *nat*
shows *nat-list-decode*(*succ*(*n*)) =
Cons(*fst*(*pair-decode*(*n*)),*nat-list-decode*(*snd*(*pair-decode*(*n*))))
⟨*proof*⟩

lemma *nat-list-decode-type* [*TC*]:
assumes *natural*: *n* ∈ *nat*
shows *nat-list-decode*(*n*) ∈ *list*(*nat*)
⟨*proof*⟩

theorem *nat-list-decode-encode*:
assumes *list*: *l* ∈ *list*(*nat*)
shows *nat-list-decode*(*nat-list-encode*(*l*)) = *l*
⟨*proof*⟩

theorem *nat-list-encode-decode*:
assumes *natural*: *n* ∈ *nat*
shows *nat-list-encode*(*nat-list-decode*(*n*)) = *n*
⟨*proof*⟩

lemma *length-nat-list-encode-le*:
assumes *list*: *l* ∈ *list*(*nat*)
shows *length*(*l*) ≤ *nat-list-encode*(*l*)
⟨*proof*⟩

lemma *length-nat-list-decode-le*:

assumes *natural*: $n \in nat$
shows $length(nat\text{-}list\text{-}decode(n)) \leq n$
⟨*proof*⟩

theorem *nat-list-encode-bij*:
$(\lambda l{\in}list(nat).\ nat\text{-}list\text{-}encode(l))$
$\in bij(list(nat),nat)$
⟨*proof*⟩

8.3 Instruction Streams

An instruction code whose action field is outside the five valid actions decodes to a no-op instruction targeting the final state. Thus decoding is defined for every natural number. The canonical codes are identified separately.

definition *encode-instruction* :: $i \Rightarrow i$ **where**
$encode\text{-}instruction(ins) \equiv pair\text{-}code(fst(ins),snd(ins))$

definition *decode-instruction* :: $i \Rightarrow i$ **where**
$decode\text{-}instruction(n) \equiv$
if $n \in nat \wedge fst(pair\text{-}decode(n)) \in action$ *then*
$pair\text{-}decode(n)$
else
$\langle nop,final\text{-}state\rangle$

definition *instruction-code* :: i **where**
$instruction\text{-}code \equiv$
$\{n{\in}nat.\ fst(pair\text{-}decode(n)) \in action\}$

lemma *instruction-code-subset-nat*:
$instruction\text{-}code \subseteq nat$
⟨*proof*⟩

lemma *encode-instruction-type* [*TC*]:
assumes *instruction*: $ins \in instruction$
shows $encode\text{-}instruction(ins) \in instruction\text{-}code$
⟨*proof*⟩

lemma *decode-instruction-type* [*TC*]:
assumes *natural*: $n \in nat$
shows $decode\text{-}instruction(n) \in instruction$
⟨*proof*⟩

theorem *decode-encode-instruction*:
assumes *instruction*: $ins \in instruction$
shows $decode\text{-}instruction(encode\text{-}instruction(ins)) = ins$
⟨*proof*⟩

theorem *encode-decode-instruction*:

assumes *code*: $n \in$ *instruction-code*
shows *encode-instruction*(*decode-instruction*(n)) = n
⟨*proof*⟩

definition *machine-code-list* :: $i \Rightarrow i$ **where**
machine-code-list(M) ≡ *map*(*encode-instruction*,M)

definition *decode-machine-list* :: $i \Rightarrow i$ **where**
decode-machine-list(l) ≡ *map*(*decode-instruction*,l)

lemma *machine-code-list-type* [TC]:
assumes *machine-M*: $M \in$ *tm-machine*
shows *machine-code-list*(M) ∈ *list*(*instruction-code*)
⟨*proof*⟩

lemma *machine-code-list-natural*:
assumes *machine-M*: $M \in$ *tm-machine*
shows *machine-code-list*(M) ∈ *list*(*nat*)
⟨*proof*⟩

lemma *decode-machine-list-type* [TC]:
assumes *codes*: $l \in$ *list*(*nat*)
shows *decode-machine-list*(l) ∈ *tm-machine*
⟨*proof*⟩

theorem *decode-machine-list-code*:
assumes *machine-M*: $M \in$ *tm-machine*
shows *decode-machine-list*(*machine-code-list*(M)) = M
⟨*proof*⟩

theorem *machine-code-list-decode*:
assumes *codes*: $l \in$ *list*(*instruction-code*)
shows *machine-code-list*(*decode-machine-list*(l)) = l
⟨*proof*⟩

8.4 Machine Numbers

Machines are encoded as finite lists of instruction numbers. Canonical codes give a bijection with machines, while the total decoder also accepts non-canonical natural numbers. The latter convention lets the self-halting predicate quantify over all natural numbers.

definition *encode-machine* :: $i \Rightarrow i$ **where**
encode-machine(M) ≡ *nat-list-encode*(*machine-code-list*(M))

definition *decode-machine* :: $i \Rightarrow i$ **where**
decode-machine(n) ≡
decode-machine-list(*nat-list-decode*(n))

definition *machine-code* :: i **where**

machine-code ≡
{*n*∈*nat*. *nat-list-decode*(*n*) ∈ *list*(*instruction-code*)}

lemma *machine-code-subset-nat*:
machine-code ⊆ *nat*
⟨*proof*⟩

lemma *encode-machine-type* [*TC*]:
assumes *machine-M*: *M* ∈ *tm-machine*
shows *encode-machine*(*M*) ∈ *machine-code*
⟨*proof*⟩

lemma *encode-machine-natural* [*TC*]:
P ∈ *tm-machine* ⟹ *encode-machine*(*P*) ∈ *nat*
⟨*proof*⟩

lemma *decode-machine-type* [*TC*]:
assumes *natural*: *n* ∈ *nat*
shows *decode-machine*(*n*) ∈ *tm-machine*
⟨*proof*⟩

theorem *decode-encode-machine*:
assumes *machine-M*: *M* ∈ *tm-machine*
shows *decode-machine*(*encode-machine*(*M*)) = *M*
⟨*proof*⟩

theorem *encode-decode-machine*:
assumes *code*: *n* ∈ *machine-code*
shows *encode-machine*(*decode-machine*(*n*)) = *n*
⟨*proof*⟩

theorem *machine-numbering-bij*:
(λ*M*∈*tm-machine*. *encode-machine*(*M*))
∈ *bij*(*tm-machine*,*machine-code*)
⟨*proof*⟩

theorem *decode-machine-surj*:
(λ*n*∈*nat*. *decode-machine*(*n*))
∈ *surj*(*nat*,*tm-machine*)
⟨*proof*⟩

end

9 Operations on Machine and List Codes

theory *Turing-Code-Operations*
imports *Turing-Coding*
begin

The operations below act directly on natural-number codes. Their correctness lemmas relate them to pairing and list operations after decoding. The arithmetic definitions are also used by the primitive-recursive certificates.

consts *pair-diagonal* :: $i \Rightarrow i$

primrec
  pair-diagonal(*0*) = *0*
  pair-diagonal(*succ*(*n*)) =
    (*if* *n* = *triangle*(*pair-diagonal*(*n*)) #+ *pair-diagonal*(*n*) *then*
      succ(*pair-diagonal*(*n*))
    else
      pair-diagonal(*n*))

lemma *pair-diagonal-type* [*TC*]:
  assumes *natural*: *n* ∈ *nat*
  shows *pair-diagonal*(*n*) ∈ *nat*
  ⟨*proof*⟩

lemma *pair-diagonal-decode*:
  assumes *natural*: *n* ∈ *nat*
  shows *pair-diagonal*(*n*) =
    fst(*pair-decode*(*n*)) #+ *snd*(*pair-decode*(*n*))
  ⟨*proof*⟩

definition *pair-right-code* :: $i \Rightarrow i$ **where**
  pair-right-code(*n*) ≡
    n #− *triangle*(*pair-diagonal*(*n*))

definition *pair-left-code* :: $i \Rightarrow i$ **where**
  pair-left-code(*n*) ≡
    pair-diagonal(*n*) #− *pair-right-code*(*n*)

lemma *pair-right-code-type* [*TC*]:
  pair-right-code(*n*) ∈ *nat*
  ⟨*proof*⟩

lemma *pair-left-code-type* [*TC*]:
  pair-left-code(*n*) ∈ *nat*
  ⟨*proof*⟩

theorem *pair-right-code-decode*:
  assumes *natural*: *n* ∈ *nat*
  shows *pair-right-code*(*n*) = *snd*(*pair-decode*(*n*))
⟨*proof*⟩

theorem *pair-left-code-decode*:
  assumes *natural*: *n* ∈ *nat*

shows *pair-left-code*(n) = *fst*(*pair-decode*(n))
⟨*proof*⟩

lemma *pair-right-code-pair-code*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
shows *pair-right-code*(*pair-code*(m,n)) = n
⟨*proof*⟩

lemma *pair-left-code-pair-code*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
shows *pair-left-code*(*pair-code*(m,n)) = m
⟨*proof*⟩

definition *code-cons* :: [i,i] ⇒ i **where**
code-cons(a,c) ≡ *succ*(*pair-code*(a,c))

definition *code-head* :: i ⇒ i **where**
code-head(c) ≡
if c = *0 then 0 else fst*(*pair-decode*(*pred*(c)))

definition *code-tail* :: i ⇒ i **where**
code-tail(c) ≡
if c = *0 then 0 else snd*(*pair-decode*(*pred*(c)))

lemma *code-cons-type* [*TC*]:
assumes *element*: $a \in nat$
and *tail*: $c \in nat$
shows *code-cons*(a,c) ∈ *nat*
⟨*proof*⟩

lemma *code-head-type* [*TC*]:
assumes *natural*: $c \in nat$
shows *code-head*(c) ∈ *nat*
⟨*proof*⟩

lemma *code-tail-type* [*TC*]:
assumes *natural*: $c \in nat$
shows *code-tail*(c) ∈ *nat*
⟨*proof*⟩

lemma *code-cons-encode* [*simp*]:
code-cons(a,*nat-list-encode*(l)) =
nat-list-encode(*Cons*(a,l))
⟨*proof*⟩

lemma *code-head-encode*:

assumes *list*: $l \in list(nat)$
shows *code-head*(*nat-list-encode*(l)) = $hd(l)$
⟨*proof*⟩

lemma *code-tail-encode*:
assumes *list*: $l \in list(nat)$
shows *code-tail*(*nat-list-encode*(l)) = *nat-list-encode*($tl(l)$)
⟨*proof*⟩

consts *code-drop* :: $[i,i] \Rightarrow i$

primrec
code-drop(0,c) = c
code-drop(*succ*(k),c) = *code-tail*(*code-drop*(k,c))

lemma *code-drop-type* [TC]:
assumes *count*: $k \in nat$
and *code*: $c \in nat$
shows *code-drop*(k,c) $\in nat$
⟨*proof*⟩

lemma *code-drop-encode*:
assumes *count*: $k \in nat$
and *list*: $l \in list(nat)$
shows *code-drop*(k,*nat-list-encode*(l)) =
nat-list-encode(*drop*(k,l))
⟨*proof*⟩

definition *code-nth* :: $[i,i] \Rightarrow i$ **where**
code-nth(k,c) ≡ *code-head*(*code-drop*(k,c))

lemma *code-nth-type* [TC]:
assumes *count*: $k \in nat$
and *code*: $c \in nat$
shows *code-nth*(k,c) $\in nat$
⟨*proof*⟩

lemma *hd-eq-nth-zero*:
assumes *list*: $l \in list(A)$
shows $hd(l)$ = *nth*(0,l)
⟨*proof*⟩

lemma *code-nth-encode*:
assumes *count*: $k \in nat$
and *list*: $l \in list(nat)$
shows *code-nth*(k,*nat-list-encode*(l)) = *nth*(k,l)

⟨*proof*⟩

definition *code-has-index* :: [*i*,*i*] ⇒ *i* **where**
code-has-index(*k*,*c*) ≡
if code-drop(*k*,*c*) = *0 then 0 else 1*

lemma *code-has-index-type* [*TC*]:
code-has-index(*k*,*c*) ∈ *nat*
⟨*proof*⟩

lemma *nat-list-encode-zero-iff*:
assumes *list*: *l* ∈ *list*(*nat*)
shows *nat-list-encode*(*l*) = *0* ⟷ *l* = []
⟨*proof*⟩

lemma *drop-eq-Nil-iff*:
assumes *count*: *k* ∈ *nat*
and *list*: *l* ∈ *list*(*A*)
shows *drop*(*k*,*l*) = [] ⟷ *length*(*l*) ≤ *k*
⟨*proof*⟩

lemma *code-has-index-encode*:
assumes *count*: *k* ∈ *nat*
and *list*: *l* ∈ *list*(*nat*)
shows *code-has-index*(*k*,*nat-list-encode*(*l*)) =
(*if k* < *length*(*l*) *then 1 else 0*)
⟨*proof*⟩

9.1 Instruction Code Normalization

definition *default-instruction-code* :: *i* **where**
default-instruction-code ≡
pair-code(*nop*,*final-state*)

definition *normalize-instruction-code* :: *i* ⇒ *i* **where**
normalize-instruction-code(*n*) ≡
if n ∈ *nat* ∧ *pair-left-code*(*n*) ∈ *action then n*
else default-instruction-code

lemma *default-instruction-code-type* [*TC*]:
default-instruction-code ∈ *nat*
⟨*proof*⟩

lemma *normalize-instruction-code-type* [*TC*]:
normalize-instruction-code(*n*) ∈ *nat*
⟨*proof*⟩

theorem *normalize-instruction-code-correct*:
normalize-instruction-code(*n*) =
encode-instruction(*decode-instruction*(*n*))
⟨*proof*⟩

end

10 Primitive-Recursive Coding

theory *Turing-Primrec*
imports
Turing-Code-Operations
ZF−Induct.Primrec
begin

This theory constructs primitive-recursive numerical operations for pairing and finite-list codes. Each certificate consists of membership in the formal class of primitive-recursive functions and an application theorem identifying its value with the corresponding code operation. Arguments form a list of natural numbers. Application equations retain an arbitrary natural-number tail to state explicitly which leading arguments the operation uses. These results do not construct a Turing machine that implements the operations.

10.1 Composition and Projection

lemma *COMP-2*:
assumes *inputs*: *l* ∈ *list*(*nat*)
shows *COMP*(*g*,[*f*,*h*]) ‘*l* = *g*‘[*f*‘*l*,*h*‘*l*]
⟨*proof*⟩

lemma *COMP-3*:
assumes *inputs*: *l* ∈ *list*(*nat*)
shows *COMP*(*g*,[*f*,*h*,*k*]) ‘*l* = *g*‘[*f*‘*l*,*h*‘*l*,*k*‘*l*]
⟨*proof*⟩

lemma *PROJ-1* [*simp*]:
assumes *first*: *x* ∈ *nat*
and *second*: *y* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *PROJ*(*1*) ‘*Cons*(*x*,*Cons*(*y*,*l*)) = *y*
⟨*proof*⟩

lemma *PROJ-2* [*simp*]:
assumes *first*: *x* ∈ *nat*
and *second*: *y* ∈ *nat*
and *third*: *z* ∈ *nat*

and *tail*: $l \in list(nat)$
shows $PROJ(2)`Cons(x,Cons(y,Cons(z,l))) = z$
⟨*proof*⟩

lemma *PROJ-3* [*simp*]:
assumes *first*: $x \in nat$
and *second*: $y \in nat$
and *third*: $z \in nat$
and *fourth*: $w \in nat$
and *tail*: $l \in list(nat)$
shows $PROJ(succ(2))`Cons(x,Cons(y,Cons(z,Cons(w,l)))) = w$
⟨*proof*⟩

10.2 Arithmetic

definition *pr-pred* :: i **where**
$pr\text{-}pred \equiv PREC(CONSTANT(0),PROJ(1))$

lemma *pr-pred-in-prim-rec* [*TC*]:
$pr\text{-}pred \in prim\text{-}rec$
⟨*proof*⟩

lemma *pr-pred-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows $pr\text{-}pred`Cons(n,l) = pred(n)$
⟨*proof*⟩

definition *pr-add* :: i **where**
$pr\text{-}add \equiv PREC(PROJ(0),SC)$

lemma *pr-add-in-prim-rec* [*TC*]:
$pr\text{-}add \in prim\text{-}rec$
⟨*proof*⟩

lemma *pr-add-apply*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
and *tail*: $l \in list(nat)$
shows $pr\text{-}add`Cons(m,Cons(n,l)) = m \#+ n$
⟨*proof*⟩

definition *pr-double* :: i **where**
$pr\text{-}double \equiv COMP(pr\text{-}add,[PROJ(0),PROJ(0)])$

lemma *pr-double-in-prim-rec* [*TC*]:
$pr\text{-}double \in prim\text{-}rec$
⟨*proof*⟩

lemma *pr-double-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-double*‘$Cons(n,l) = n \#+ n$
⟨*proof*⟩

definition *pr-reverse-diff* :: i **where**
pr-reverse-diff ≡
$PREC(PROJ(0),COMP(pr\text{-}pred,[PROJ(0)]))$

lemma *pr-reverse-diff-in-prim-rec* [*TC*]:
pr-reverse-diff ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-reverse-diff-apply*:
assumes *subtrahend*: $n \in nat$
and *minuend*: $m \in nat$
and *tail*: $l \in list(nat)$
shows *pr-reverse-diff*‘$Cons(n,Cons(m,l)) = m \#- n$
⟨*proof*⟩

definition *pr-diff* :: i **where**
pr-diff ≡
$COMP(pr\text{-}reverse\text{-}diff,[PROJ(1),PROJ(0)])$

lemma *pr-diff-in-prim-rec* [*TC*]:
pr-diff ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-diff-apply*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-diff*‘$Cons(m,Cons(n,l)) = m \#- n$
⟨*proof*⟩

lemma *succ-diff-succ*:
assumes *index*: $j \in nat$
and *total*: $n \in nat$
and *bounded*: $succ(j) \leq n$
shows $succ(n \#- succ(j)) = n \#- j$
⟨*proof*⟩

10.3 Conditionals

definition *pr-is-zero* :: i **where**
pr-is-zero ≡ $PREC(CONSTANT(1),CONSTANT(0))$

lemma *pr-is-zero-in-prim-rec* [*TC*]:

pr-is-zero ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-is-zero-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-is-zero*‘*Cons*(n,l) = (*if* $n = 0$ *then* 1 *else* 0)
⟨*proof*⟩

definition *pr-positive* :: i **where**
pr-positive ≡ $PREC(CONSTANT(0),CONSTANT(1))$

lemma *pr-positive-in-prim-rec* [TC]:
pr-positive ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-positive-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-positive*‘*Cons*(n,l) = (*if* $n = 0$ *then* 0 *else* 1)
⟨*proof*⟩

definition *pr-if-zero* :: i **where**
pr-if-zero ≡ $PREC(PROJ(0),PROJ(succ(2)))$

lemma *pr-if-zero-in-prim-rec* [TC]:
pr-if-zero ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-if-zero-apply*:
assumes *condition*: $c \in nat$
and *zero*: $z \in nat$
and *successor*: $s \in nat$
and *tail*: $l \in list(nat)$
shows *pr-if-zero*‘*Cons*(c,*Cons*(z,*Cons*(s,l))) =
(*if* $c = 0$ *then* z *else* s)
⟨*proof*⟩

definition *pr-le* :: i **where**
pr-le ≡ $COMP(pr\text{-}is\text{-}zero,[pr\text{-}diff])$

lemma *pr-le-in-prim-rec* [TC]:
pr-le ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-le-apply*:
assumes *first*: $m \in nat$
and *second*: $n \in nat$
and *tail*: $l \in list(nat)$

shows *pr-le‘Cons*(*m*,*Cons*(*n*,*l*)) = (*if m* ≤ *n then 1 else 0*)
⟨*proof*⟩

10.4 Cantor Pairing

definition *pr-triangle-step* :: *i* **where**
pr-triangle-step ≡
COMP(*pr-add*,[*PROJ*(*0*),*COMP*(*SC*,[*PROJ*(*1*)])])

lemma *pr-triangle-step-in-prim-rec* [*TC*]:
pr-triangle-step ∈ *prim-rec*
⟨*proof*⟩

definition *pr-triangle* :: *i* **where**
pr-triangle ≡ *PREC*(*CONSTANT*(*0*),*pr-triangle-step*)

lemma *pr-triangle-in-prim-rec* [*TC*]:
pr-triangle ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-triangle-apply*:
assumes *natural*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-triangle‘Cons*(*n*,*l*) = *triangle*(*n*)
⟨*proof*⟩

definition *pr-pair-code* :: *i* **where**
pr-pair-code ≡
COMP(*pr-add*,[*COMP*(*pr-triangle*,[*pr-add*]),*PROJ*(*1*)])

theorem *pr-pair-code-in-prim-rec* [*TC*]:
pr-pair-code ∈ *prim-rec*
⟨*proof*⟩

theorem *pr-pair-code-apply*:
assumes *first*: *m* ∈ *nat*
and *second*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-pair-code‘Cons*(*m*,*Cons*(*n*,*l*)) = *pair-code*(*m*,*n*)
⟨*proof*⟩

10.5 Cantor Projections

definition *pr-equal-core* :: *i* **where**
pr-equal-core ≡ *COMP*(*pr-add*,[*pr-diff*,*pr-reverse-diff*])

lemma *pr-equal-core-in-prim-rec* [*TC*]:
pr-equal-core ∈ *prim-rec*
⟨*proof*⟩

definition *pr-equal* :: *i* **where**
pr-equal ≡ *COMP*(*pr-is-zero*,[*pr-equal-core*])

lemma *pr-equal-in-prim-rec* [*TC*]:
pr-equal ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-equal-apply*:
assumes *first*: *m* ∈ *nat*
and *second*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-equal*‘*Cons*(*m*,*Cons*(*n*,*l*)) = (*if m* = *n then 1 else 0*)
⟨*proof*⟩

definition *pr-diagonal-end* :: *i* **where**
pr-diagonal-end ≡
COMP(*pr-add*,[*COMP*(*pr-triangle*,[*PROJ*(*0*)]),*PROJ*(*0*)])

lemma *pr-diagonal-end-in-prim-rec* [*TC*]:
pr-diagonal-end ∈ *prim-rec*
⟨*proof*⟩

definition *pr-pair-diagonal-step* :: *i* **where**
pr-pair-diagonal-step ≡
COMP(*pr-if-zero*,
[*COMP*(*pr-equal*,[*PROJ*(*1*),*pr-diagonal-end*]),
PROJ(*0*),
COMP(*SC*,[*PROJ*(*0*)])])

lemma *pr-pair-diagonal-step-in-prim-rec* [*TC*]:
pr-pair-diagonal-step ∈ *prim-rec*
⟨*proof*⟩

definition *pr-pair-diagonal* :: *i* **where**
pr-pair-diagonal ≡
PREC(*CONSTANT*(*0*),*pr-pair-diagonal-step*)

lemma *pr-pair-diagonal-in-prim-rec* [*TC*]:
pr-pair-diagonal ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-pair-diagonal-apply*:
assumes *natural*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-pair-diagonal*‘*Cons*(*n*,*l*) = *pair-diagonal*(*n*)
⟨*proof*⟩

definition *pr-pair-right* :: *i* **where**
pr-pair-right ≡

$COMP(pr\text{-}diff,[PROJ(0),COMP(pr\text{-}triangle,[pr\text{-}pair\text{-}diagonal])])$

lemma *pr-pair-right-in-prim-rec* [*TC*]:
pr-pair-right ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-pair-right-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-pair-right*‘*Cons*$(n,l) = snd(pair\text{-}decode(n))$
⟨*proof*⟩

definition *pr-pair-left* :: *i* **where**
pr-pair-left ≡
$COMP(pr\text{-}diff,[pr\text{-}pair\text{-}diagonal,pr\text{-}pair\text{-}right])$

lemma *pr-pair-left-in-prim-rec* [*TC*]:
pr-pair-left ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-pair-left-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-pair-left*‘*Cons*$(n,l) = fst(pair\text{-}decode(n))$
⟨*proof*⟩

corollary *pr-pair-right-code-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-pair-right*‘*Cons*$(n,l) = pair\text{-}right\text{-}code(n)$
⟨*proof*⟩

corollary *pr-pair-left-code-apply*:
assumes *natural*: $n \in nat$
and *tail*: $l \in list(nat)$
shows *pr-pair-left*‘*Cons*$(n,l) = pair\text{-}left\text{-}code(n)$
⟨*proof*⟩

10.6 Coded Natural Lists

definition *pr-list-cons* :: *i* **where**
$pr\text{-}list\text{-}cons \equiv COMP(SC,[pr\text{-}pair\text{-}code])$

lemma *pr-list-cons-in-prim-rec* [*TC*]:
pr-list-cons ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-list-cons-apply*:
assumes *element*: $a \in nat$

and *tail-code*: $c \in nat$
and *tail*: $l \in list(nat)$
shows *pr-list-cons*`*Cons*$(a,Cons(c,l)) = code\text{-}cons(a,c)$
⟨*proof*⟩

definition *pr-list-head* :: i **where**
pr-list-head ≡
$COMP(pr\text{-}if\text{-}zero,$
$[PROJ(0),CONSTANT(0),$
$COMP(pr\text{-}pair\text{-}left,[COMP(pr\text{-}pred,[PROJ(0)])])])$

lemma *pr-list-head-in-prim-rec* [*TC*]:
pr-list-head ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-list-head-apply*:
assumes *natural*: $c \in nat$
and *tail*: $l \in list(nat)$
shows *pr-list-head*`*Cons*$(c,l) = code\text{-}head(c)$
⟨*proof*⟩

definition *pr-list-tail* :: i **where**
pr-list-tail ≡
$COMP(pr\text{-}if\text{-}zero,$
$[PROJ(0),CONSTANT(0),$
$COMP(pr\text{-}pair\text{-}right,[COMP(pr\text{-}pred,[PROJ(0)])])])$

lemma *pr-list-tail-in-prim-rec* [*TC*]:
pr-list-tail ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-list-tail-apply*:
assumes *natural*: $c \in nat$
and *tail*: $l \in list(nat)$
shows *pr-list-tail*`*Cons*$(c,l) = code\text{-}tail(c)$
⟨*proof*⟩

definition *pr-list-drop* :: i **where**
pr-list-drop ≡
$PREC(PROJ(0),COMP(pr\text{-}list\text{-}tail,[PROJ(0)]))$

lemma *pr-list-drop-in-prim-rec* [*TC*]:
pr-list-drop ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-list-drop-apply*:
assumes *count*: $k \in nat$
and *code*: $c \in nat$
and *tail*: $l \in list(nat)$

shows *pr-list-drop‘Cons*(k,*Cons*(c,l)) = *code-drop*(k,c)
⟨*proof*⟩

definition *pr-list-nth* :: i **where**
pr-list-nth ≡ *COMP*(*pr-list-head*,[*pr-list-drop*])

lemma *pr-list-nth-in-prim-rec* [*TC*]:
pr-list-nth ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-list-nth-apply*:
assumes *count*: $k \in nat$
and *code*: $c \in nat$
and *tail*: $l \in list(nat)$
shows *pr-list-nth‘Cons*(k,*Cons*(c,l)) = *code-nth*(k,c)
⟨*proof*⟩

definition *pr-list-has-index* :: i **where**
pr-list-has-index ≡
COMP(*pr-positive*,[*pr-list-drop*])

lemma *pr-list-has-index-in-prim-rec* [*TC*]:
pr-list-has-index ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-list-has-index-apply*:
assumes *count*: $k \in nat$
and *code*: $c \in nat$
and *tail*: $l \in list(nat)$
shows *pr-list-has-index‘Cons*(k,*Cons*(c,l)) = *code-has-index*(k,c)
⟨*proof*⟩

end

11 Primitive-Recursive Instruction Normalization

theory *Turing-Instruction-Normalization*
imports *Turing-Primrec*
begin

Invalid action codes denote the no-op instruction entering the final state. Normalization and its primitive-recursive certificate depend only on coding operations.

lemma *action-iff-le-nop*:
assumes *natural*: $a \in nat$
shows $a \in action \longleftrightarrow a \leq nop$
⟨*proof*⟩

definition *pr-valid-action* :: *i* **where**
pr-valid-action ≡
COMP(*pr-le*,[*PROJ*(*0*),*CONSTANT*(*nop*)])

lemma *pr-valid-action-in-prim-rec* [*TC*]:
pr-valid-action ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-valid-action-apply*:
assumes *action-code*: *a* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-valid-action*‘*Cons*(*a*,*l*) =
(*if a* ∈ *action then 1 else 0*)
⟨*proof*⟩

definition *pr-valid-instruction-code* :: *i* **where**
pr-valid-instruction-code ≡
COMP(*pr-valid-action*,[*pr-pair-left*])

lemma *pr-valid-instruction-code-in-prim-rec* [*TC*]:
pr-valid-instruction-code ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-valid-instruction-code-apply*:
assumes *instruction-number*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-valid-instruction-code*‘*Cons*(*n*,*l*) =
(*if pair-left-code*(*n*) ∈ *action then 1 else 0*)
⟨*proof*⟩

definition *pr-normalize-instruction-code* :: *i* **where**
pr-normalize-instruction-code ≡
COMP(*pr-if-zero*,
[*pr-valid-instruction-code*,
CONSTANT(*default-instruction-code*),
PROJ(*0*)])

lemma *pr-normalize-instruction-code-in-prim-rec* [*TC*]:
pr-normalize-instruction-code ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-normalize-instruction-code-apply*:
assumes *instruction-number*: *n* ∈ *nat*
and *tail*: *l* ∈ *list*(*nat*)
shows *pr-normalize-instruction-code*‘*Cons*(*n*,*l*) =
normalize-instruction-code(*n*)
⟨*proof*⟩

end

12 Finite Witnesses in Transitive Models

theory *Turing-Model-Witnesses*
imports
Turing-Machine
Independence-CH.ZF-Trans-Interpretations
begin

definition *transitive-zf-model* :: $i \Rightarrow o$ **where**
$\mathit{transitive\text{-}zf\text{-}model}(M) \equiv \mathit{Transset}(M) \wedge M \models \mathit{ZF}$

Finite machines, inputs, configurations, and canonical finite traces belong to each transitive ZF set model. In *transitive-zf-model*, *ZF* is the internalised axiom set from *Independence-CH.Internal-ZFC-Axioms*. The predicate requires satisfaction of those axioms by the given set. Choice is not required for finite object closure or finite runs.

12.1 Finite object closure

context *M-trivial*
begin

lemma *tm-list-closed*:
assumes *xs*: $xs \in \mathit{list}(A)$ **and** *elements*: $\bigwedge a.\ a \in A \Longrightarrow M(a)$
shows $M(xs)$
⟨*proof*⟩

lemma *tm-symbol-closed*: $a \in \mathit{symbol} \Longrightarrow M(a)$
⟨*proof*⟩

lemma *tm-instruction-closed*: $\mathit{ins} \in \mathit{instruction} \Longrightarrow M(\mathit{ins})$
⟨*proof*⟩

lemma *tm-machine-closed*: $P \in \mathit{tm\text{-}machine} \Longrightarrow M(P)$
⟨*proof*⟩

lemma *tm-input-closed*: $x \in \mathit{list}(\mathit{symbol}) \Longrightarrow M(x)$
⟨*proof*⟩

lemma *tm-configuration-closed*: $c \in \mathit{configuration} \Longrightarrow M(c)$
⟨*proof*⟩

lemma *tm-finite-lambda-closed*:
assumes *n*: $n \in \mathit{nat}$ **and** *values*: $\bigwedge k.\ k \in n \Longrightarrow M(f(k))$
shows $M(\lambda k \in n.\ f(k))$
⟨*proof*⟩

Finite traces are built from ordered pairs by finite induction. The closure theorem *tm-finite-lambda-closed* requires only that each value belongs to the model. It does not require the whole function to have been given internally.

lemma *tm-trace-closed*:
assumes *P*: $P \in$ *tm-machine* **and** *c*: $c \in$ *configuration* **and** *n*: $n \in nat$
shows $M(\lambda k \in succ(n).\ steps(P,c,k))$
⟨*proof*⟩
end

12.2 Transitive model interpretations

lemma *transitive-zf-M-ZF1-trans*:
assumes *model*: *transitive-zf-model*(A)
shows *M-ZF1-trans*(A)
⟨*proof*⟩

lemma *transitive-zf-M-trivial*:
transitive-zf-model$(A) \Longrightarrow$ *M-trivial*$(\#\#A)$
⟨*proof*⟩

lemma *transitive-zf-M-trancl*:
transitive-zf-model$(A) \Longrightarrow$ *M-trancl*$(\#\#A)$
⟨*proof*⟩

12.3 Machine objects in a set model

theorem *transitive-zf-contains-machine*:
⟦*transitive-zf-model*(A); $P \in$ *tm-machine*⟧ $\Longrightarrow$ $P \in A$
⟨*proof*⟩

theorem *transitive-zf-contains-configuration*:
⟦*transitive-zf-model*(A); $c \in$ *configuration*⟧ $\Longrightarrow$ $c \in A$
⟨*proof*⟩

theorem *transitive-zf-contains-trace*:
⟦*transitive-zf-model*(A); $P \in$ *tm-machine*; $c \in$ *configuration*; $n \in nat$⟧ $\Longrightarrow$
$(\lambda k \in succ(n).\ steps(P,c,k)) \in A$
⟨*proof*⟩

end

13 Natural Membership in Transitive Models

theory *Nat-Membership*
imports *Turing-Model-Witnesses*
begin

The formula expresses membership in the standard natural numbers. Transitivity identifies the model's natural numbers with the external set *nat*.

definition *nat-member-fm* :: $i \Rightarrow i$ **where**
nat-member-fm$(i) \equiv$ *Exists*(*And*(*omega-fm*(*0*),*Member*(*succ*(*i*),*0*)))

The existentially bound set at index 0 is the model's omega. The original variable at index i shifts to $succ(i)$, and membership in that set expresses membership in the external natural numbers.

lemma *nat-member-fm-type* [*TC*]:
  $i \in nat \Longrightarrow$ *nat-member-fm*(i) $\in$ *formula*
  ⟨*proof*⟩

theorem *sats-nat-member-fm*:
  assumes *model*: *M-ZF1-trans*(A) **and** *env*: *env* $\in$ *list*(A)
    and i: $i \in nat$
  shows *sats*(A,*nat-member-fm*(i),*env*) $\longleftrightarrow$ *nth*(i,*env*) $\in$ *nat*
⟨*proof*⟩

end

14 Internal Halting Formula

theory *Turing-Halting-Formula*
  imports *Nat-Membership*
begin

This theory constructs an explicit first-order formula *halt-fm* of set theory with two free variables and proves that, in every transitive set model of ZF, it holds of a machine P and an input x exactly when P halts on x (theorem *transitive-model-halt-fm-iff*).

The formulas for list traversal and machine execution constrain finitely many applications of an existential witness set. They do not require every witness to be a finite function. An external run supplies a canonical finite function belonging to the model by *transitive-zf-contains-trace*. Conversely, induction recovers the required finite sequence from any internal witness.

The formulas *drop-fm* and *fetch-fm* are internal components. The public interface consists of the halting satisfaction theorems. The library also supplies *nth-abs* in *M-datatypes*. That locale assumes replacement for list and formula constructions and list indexing. Here finite traversal witnesses are verified in *M-ZF1-trans*, without interpreting *M-datatypes*.

14.1 Names for natural numbers

definition *nat-name-fm* :: $i \Rightarrow i$ **where**
  nat-name-fm(n) $\equiv$ *nat-rec*(n, *empty-fm*(*0*), $\lambda k\ p$. *Exists*(*And*(p,
    succ-fm(*0*,*1*))))

The numeral name defines the value at index 0. Its successor clause binds the predecessor at index 0 and shifts the value being named to index 1. The formula *const-fm*(k,i) below transports this name to the selected environment index i.

lemma *nat-name-fm-0* [*simp*]: *nat-name-fm*(*0*) = *empty-fm*(*0*)
⟨*proof*⟩

lemma *nat-name-fm-succ* [*simp*]:
n ∈ *nat* ⟹ *nat-name-fm*(*succ*(*n*)) =
Exists(*And*(*nat-name-fm*(*n*), *succ-fm*(*0*,*1*)))
⟨*proof*⟩

lemma *nat-name-fm-type* [*TC*]: *n* ∈ *nat* ⟹ *nat-name-fm*(*n*) ∈ *formula*
⟨*proof*⟩

lemma *arity-nat-name-fm*: *n* ∈ *nat* ⟹ *arity*(*nat-name-fm*(*n*)) = *1*
⟨*proof*⟩

lemma *sats-nat-name-fm*:
assumes *model*: *M-ZF1-trans*(*A*) **and** *n*: *n* ∈ *nat*
shows ⋀*x env*. *x* ∈ *A* ⟹ *env* ∈ *list*(*A*) ⟹
(*A*, *Cons*(*x*,*env*) ⊨ *nat-name-fm*(*n*)) ⟷ *x* = *n*
⟨*proof*⟩

definition *const-fm* :: [*i*,*i*] ⇒ *i* **where**
const-fm(*k*,*i*) ≡ *Exists*(*And*(*nat-name-fm*(*k*), *Equal*(*0*,*succ*(*i*))))

lemma *const-fm-type* [*TC*]: ⟦*k* ∈ *nat*; *i* ∈ *nat*⟧ ⟹ *const-fm*(*k*,*i*) ∈ *formula*
⟨*proof*⟩

lemma *sats-const-fm*:
assumes *model*: *M-ZF1-trans*(*A*) **and** *k*: *k* ∈ *nat* **and** *i*: *i* ∈ *nat*
and *env*: *env* ∈ *list*(*A*)
shows (*A*, *env* ⊨ *const-fm*(*k*,*i*)) ⟷ *nth*(*i*,*env*) = *k*
⟨*proof*⟩

14.2 Model and tape updates

The satisfaction proofs use *M-ZF1-trans*, a transitive model of Zermelo set theory with the four replacement instances specified by the imported locale. A transitive ZF set model interprets this locale. No minimality claim about these assumptions is made.

context *M-ZF1-trans*
begin

lemma *tm-const* [*simp*]:
⟦*k* ∈ *nat*; *i* ∈ *nat*; *env* ∈ *list*(*M*)⟧ ⟹
(*M*, *env* ⊨ *const-fm*(*k*,*i*)) ⟷ *nth*(*i*,*env*) = *k*
⟨*proof*⟩

lemma *list-in-A*:
assumes *l*: *l* ∈ *list*(*B*) **and** *el*: ⋀*a*. *a* ∈ *B* ⟹ *a* ∈ *M*
shows *l* ∈ *M*

⟨*proof*⟩

lemma *nat-in-A* [*simp*]: $n \in nat \Longrightarrow n \in M$
⟨*proof*⟩

lemma *pair-in-A* [*simp*]: $\langle a,b\rangle \in M \longleftrightarrow a \in M \wedge b \in M$
⟨*proof*⟩

lemma *Nil-in-A* [*simp*]: $[] \in M$
⟨*proof*⟩

lemma *Cons-in-A* [*simp*]: $Cons(a,l) \in M \longleftrightarrow a \in M \wedge l \in M$
⟨*proof*⟩

lemma *symbols-in-A* [*simp*]: $l \in list(symbol) \Longrightarrow l \in M$
⟨*proof*⟩

lemma *hd'-list* [*simp*]: $l \in list(B) \Longrightarrow hd'(l) = hd(l)$
⟨*proof*⟩

lemma *tl'-list* [*simp*]: $l \in list(B) \Longrightarrow tl'(l) = tl(l)$
⟨*proof*⟩

lemma *hd-symbol-in-A* [*simp*]: $l \in list(symbol) \Longrightarrow hd(l) \in M$
⟨*proof*⟩

lemma *tl-symbols* [*simp*]: $l \in list(symbol) \Longrightarrow tl(l) \in list(symbol)$
⟨*proof*⟩

lemma *tl-symbols-in-A* [*simp*]: $l \in list(symbol) \Longrightarrow tl(l) \in M$
⟨*proof*⟩

end

definition *upd-fm* :: $[i,i,i,i,i] \Rightarrow i$ **where**
upd-fm$(a,l,r,l2,r2) \equiv$
Or(*And*(*const-fm*(0,*a*), *And*(*Equal*(*l2*,*l*),
Exists(*Exists*(*And*(*const-fm*(0,1), *And*(*tl-fm*(*r*#+2,0),
Cons-fm(1,0,*r2*#+2)))))),
Or(*And*(*const-fm*(1,*a*), *And*(*Equal*(*l2*,*l*),
Exists(*Exists*(*And*(*const-fm*(1,1), *And*(*tl-fm*(*r*#+2,0),
Cons-fm(1,0,*r2*#+2)))))),
Or(*And*(*const-fm*(2,*a*),
Exists(*Exists*(*And*(*tl-fm*(*l*#+2,0), *And*(*hd-fm*(*l*#+2,1),
And(*Equal*(*l2*#+2,0), *Cons-fm*(1,*r*#+2,*r2*#+2)))))),
Or(*And*(*const-fm*(3,*a*),
Exists(*Exists*(*And*(*hd-fm*(*r*#+2,1), *And*(*tl-fm*(*r*#+2,0),
And(*Cons-fm*(1,*l*#+2,*l2*#+2), *Equal*(*r2*#+2,0)))))),
And(*Neg*(*Or*(*const-fm*(0,*a*), *Or*(*const-fm*(1,*a*), *Or*(*const-fm*(2,*a*),

const-fm(3,*a*)))))),
And(*Equal*(*l2*,*l*), *Equal*(*r2*,*r*)))))))

lemma *upd-fm-type* [*TC*]:
⟦*a* ∈ *nat*; *l* ∈ *nat*; *r* ∈ *nat*; *l2* ∈ *nat*; *r2* ∈ *nat*⟧
⟹ *upd-fm*(*a*,*l*,*r*,*l2*,*r2*) ∈ *formula*
⟨*proof*⟩

context *M-ZF1-trans*
begin

lemma *sats-upd-fm*:
assumes *idx*: *ia* ∈ *nat* *il* ∈ *nat* *ir* ∈ *nat* *il2* ∈ *nat* *ir2* ∈ *nat*
and *env*: *env* ∈ *list*(*M*)
and *a*: *nth*(*ia*,*env*) = *a*
and *l*: *nth*(*il*,*env*) = *l* *l* ∈ *list*(*symbol*)
and *r*: *nth*(*ir*,*env*) = *r* *r* ∈ *list*(*symbol*)
and *l2*: *nth*(*il2*,*env*) = *l2* *l2* ∈ *M*
and *r2*: *nth*(*ir2*,*env*) = *r2* *r2* ∈ *M*
shows (*M*, *env* ⊨ *upd-fm*(*ia*,*il*,*ir*,*il2*,*ir2*)) ⟷ ⟨*l2*,*r2*⟩ = *tm-update*(*a*,⟨*l*,*r*⟩)
⟨*proof*⟩

end

14.3 Iterated tails

definition *drop-fm* :: [*i*,*i*,*i*] ⇒ *i* **where**
drop-fm(*n*,*l*,*d*) ≡
Exists(*And*(*Exists*(*And*(*empty-fm*(0), *fun-apply-fm*(1,0,*l*#+2))),
And(*Forall*(*Implies*(*Member*(0,*n*#+2),
Exists(*Exists*(*Exists*(*And*(*succ-fm*(3,2),
And(*fun-apply-fm*(4,3,1), *And*(*fun-apply-fm*(4,2,0),
tl-fm(1,0)))))))))),
fun-apply-fm(0,*n*#+1,*d*#+1))))

The witness for *drop-fm* records successive list tails. At the outer existential quantifier its index is 0. Under the bounded universal quantifier and three further existential quantifiers, the environment has the prefix
[*f*‘*succ*(*k*),*f*‘*k*,*succ*(*k*),*k*,*f*].
The recurrence relates entries 1 and 0 by *tl-fm*. The initial and final applications select the original list and the required tail.

lemma *drop-fm-type* [*TC*]:
⟦*n* ∈ *nat*; *l* ∈ *nat*; *d* ∈ *nat*⟧ ⟹ *drop-fm*(*n*,*l*,*d*) ∈ *formula*
⟨*proof*⟩

definition *drop-witness* :: [*i*,*i*,*i*,*i*] ⇒ *o* **where**
drop-witness(*f*,*n*,*l*,*d*) ≡ *f*‘0 = *l* ∧ (∀ *k*∈*n*. *f*‘*succ*(*k*) =
tl′(*f*‘*k*)) ∧ *f*‘*n* = *d*

definition *tm-drop-tail* :: $[i,i] \Rightarrow i$ **where**
  $tm\text{-}drop\text{-}tail(n,l) \equiv (\lambda x.\ tl'(x))^{\frown}n\ (l)$

lemma *tm-drop-tail-0* [*simp*]: $tm\text{-}drop\text{-}tail(0,l) = l$
  ⟨*proof*⟩

lemma *tm-drop-tail-succ* [*simp*]:
  $n \in nat \Longrightarrow$
    $tm\text{-}drop\text{-}tail(succ(n),l) = tl'(tm\text{-}drop\text{-}tail(n,l))$
  ⟨*proof*⟩

lemma *tm-drop-tail-list*:
  assumes *l*: $l \in list(B)$ **and** *n*: $n \in nat$
  shows $tm\text{-}drop\text{-}tail(n,l) = drop(n,l)$
  ⟨*proof*⟩

context *M-ZF1-trans*
begin

lemma *sats-drop-fm-witness*:
  assumes *idx*: $jn \in nat\ il \in nat\ jd \in nat$ **and** *env*: $env \in list(M)$
    and *n*: $nth(jn,env) = n\ n \in nat$
    and *l*: $nth(il,env) = l\ l \in M$
    and *d*: $nth(jd,env) = d\ d \in M$
  shows $(M,\ env \models drop\text{-}fm(jn,il,jd)) \longleftrightarrow (\exists f{\in}M.\ drop\text{-}witness(f,n,l,d))$
⟨*proof*⟩

lemma *tl'-in-A* [*simp*]: $x \in M \Longrightarrow tl'(x) \in M$
⟨*proof*⟩

lemma *tm-drop-tail-in-A* [*simp*]:
  assumes *l*: $l \in M$ **and** *n*: $n \in nat$
  shows $tm\text{-}drop\text{-}tail(n,l) \in M$
  ⟨*proof*⟩

lemma *drop-witness-iff*:
  assumes *n*: $n \in nat$ **and** *l*: $l \in M$ **and** *d*: $d \in M$
  shows $(\exists f{\in}M.\ drop\text{-}witness(f,n,l,d)) \longleftrightarrow d = tm\text{-}drop\text{-}tail(n,l)$
⟨*proof*⟩

lemma *sats-drop-fm* [*simp*]:
  assumes *idx*: $jn \in nat\ il \in nat\ jd \in nat$ **and** *env*: $env \in list(M)$
    and *n*: $nth(jn,env) \in nat$ **and** *l*: $nth(il,env) \in M$ **and** *d*:
      $nth(jd,env) \in M$
  shows $(M,\ env \models drop\text{-}fm(jn,il,jd)) \longleftrightarrow$
    $nth(jd,env) = tm\text{-}drop\text{-}tail(nth(jn,env),nth(il,env))$
  ⟨*proof*⟩

end

14.4 Fetching an instruction

lemma *nth-via-drop*:
  assumes *P*: $P \in list(B)$ **and** *s*: $s \in nat$
  shows $(if\ s < length(P)\ then\ nth(s,P)\ else\ c) =$
    $(if\ drop(s,P) = []\ then\ c\ else\ hd(drop(s,P)))$
⟨*proof*⟩

lemma *fetch-via-drop*:
  assumes *P*: $P \in list(B)$ **and** *q*: $q \in nat$ **and** *b*: $b \in nat$
  shows $fetch(P,q,b) = (if\ q = 0\ then\ \langle nop,0\rangle$
      $else\ if\ drop(b,\ drop(pred(q),\ drop(pred(q),P))) = []\ then\ \langle nop,0\rangle$
      $else\ hd(drop(b,\ drop(pred(q),\ drop(pred(q),P)))))$
⟨*proof*⟩

definition *nop-instr-fm* :: $i \Rightarrow i$ **where**
  $nop\text{-}instr\text{-}fm(i) \equiv Exists(Exists(And(const\text{-}fm(nop,1),$
    $And(const\text{-}fm(0,0),\ pair\text{-}fm(1,0,i\#+2)))))$

lemma *nop-instr-fm-type* [*TC*]: $i \in nat \Longrightarrow nop\text{-}instr\text{-}fm(i) \in formula$
  ⟨*proof*⟩

definition *fetch-fm* :: $[i,i,i,i] \Rightarrow i$ **where**
  $fetch\text{-}fm(p,q,b,ins) \equiv$
    $Or(And(const\text{-}fm(0,q),\ nop\text{-}instr\text{-}fm(ins)),$
      $And(Neg(const\text{-}fm(0,q)),$
        $Exists(And(succ\text{-}fm(0,q\#+1),$
          $Exists(And(drop\text{-}fm(1,p\#+2,0),$
            $Exists(And(drop\text{-}fm(2,1,0),$
              $Exists(And(drop\text{-}fm(b\#+4,1,0),$
                $Or(And(Nil\text{-}fm(0),\ nop\text{-}instr\text{-}fm(ins\#+4)),$
                  $And(Neg(Nil\text{-}fm(0)),\ hd\text{-}fm(0,ins\#+4))))))))))))$

State 0 selects the no-op instruction. For a positive state q, list traversal drops $q - 1$ instructions twice and then drops the scanned symbol number. This selects index $2 * (q - 1) + b$ of the instruction list. An exhausted list also selects the no-op instruction. These cases match the total external function *fetch*.

lemma *fetch-fm-type* [*TC*]:
  $[\![p \in nat;\ q \in nat;\ b \in nat;\ ins \in nat]\!] \Longrightarrow fetch\text{-}fm(p,q,b,ins) \in formula$
  ⟨*proof*⟩

context *M-ZF1-trans*
begin

lemma *nth-in-A* [*simp*]: $env \in list(M) \Longrightarrow i \in nat \Longrightarrow nth(i,env) \in M$
⟨*proof*⟩

lemma *nop-in-A* [*simp*]: $nop \in M$

⟨*proof*⟩

lemma *sats-nop-instr-fm* [*simp*]:
⟦*i* ∈ *nat*; *env* ∈ *list*(*M*)⟧ ⟹ (*M*, *env* ⊨ *nop-instr-fm*(*i*)) ⟷ *nth*(*i*,*env*) = ⟨*nop*,*0*⟩
⟨*proof*⟩

lemma *machine-in-A* [*simp*]: *P* ∈ *tm-machine* ⟹ *P* ∈ *M*
⟨*proof*⟩

lemma *sats-fetch-fm*:
assumes *idx*: *ip* ∈ *nat* *iq* ∈ *nat* *ib* ∈ *nat* *ii* ∈ *nat* **and**
env: *env* ∈ *list*(*M*)
and *P*: *nth*(*ip*,*env*) = *P* *P* ∈ *tm-machine*
and *q*: *nth*(*iq*,*env*) = *q* *q* ∈ *nat*
and *b*: *nth*(*ib*,*env*) = *b* *b* ∈ *nat*
and *ins*: *nth*(*ii*,*env*) = *ins* *ins* ∈ *M*
shows (*M*, *env* ⊨ *fetch-fm*(*ip*,*iq*,*ib*,*ii*)) ⟷ *ins* = *fetch*(*P*,*q*,*b*)
⟨*proof*⟩

lemma *sats-upd-fm-simp* [*simp*]:
⟦*ia* ∈ *nat*; *il* ∈ *nat*; *ir* ∈ *nat*; *il2* ∈ *nat*; *ir2* ∈ *nat*; *env* ∈ *list*(*M*);
nth(*il*,*env*) ∈ *list*(*symbol*); *nth*(*ir*,*env*) ∈ *list*(*symbol*)⟧ ⟹
(*M*, *env* ⊨ *upd-fm*(*ia*,*il*,*ir*,*il2*,*ir2*)) ⟷ ⟨*nth*(*il2*,*env*),*nth*(*ir2*,*env*)⟩ =
tm-update(*nth*(*ia*,*env*),⟨*nth*(*il*,*env*),*nth*(*ir*,*env*)⟩)
⟨*proof*⟩

lemma *sats-fetch-fm-simp* [*simp*]:
⟦*ip* ∈ *nat*; *iq* ∈ *nat*; *ib* ∈ *nat*; *ii* ∈ *nat*; *env* ∈ *list*(*M*);
nth(*ip*,*env*) ∈ *tm-machine*; *nth*(*iq*,*env*) ∈ *nat*; *nth*(*ib*,*env*) ∈ *nat*⟧ ⟹
(*M*, *env* ⊨ *fetch-fm*(*ip*,*iq*,*ib*,*ii*)) ⟷
nth(*ii*,*env*) = *fetch*(*nth*(*ip*,*env*),*nth*(*iq*,*env*),*nth*(*ib*,*env*))
⟨*proof*⟩

end

14.5 One step

definition *step-fm* :: [*i*,*i*,*i*] ⇒ *i* **where**
step-fm(*p*,*c*,*d*) ≡
Exists(*Exists*(*Exists*(*Exists*(*Exists*(*Exists*(
Exists(*Exists*(*Exists*(*Exists*(*Exists*(
And(*pair-fm*(*10*,*9*,*c#+11*),
And(*pair-fm*(*8*,*7*,*9*),
And(*hd-fm*(*7*,*6*),
And(*fetch-fm*(*p#+11*,*10*,*6*,*5*),
And(*pair-fm*(*4*,*3*,*5*),
And(*upd-fm*(*4*,*8*,*7*,*2*,*1*),
And(*pair-fm*(*2*,*1*,*0*),

pair-fm(*3*,*0*,*d#+11*)))))))))))))))))))

At the innermost conjunction, the eleven existentially bound variables form the environment prefix [t',r',l',q',*a*,*i*,*b*,*r*,*l*,*t*,*q*], in index order from 0 to 10. The original configuration is $\langle q,t\rangle$ with $t = \langle l,r\rangle$. The scanned symbol is $b = hd(r)$, the fetched instruction is $i = \langle a,q'\rangle$, and the updated tape is $t' = \langle l',r'\rangle$. The free indices for the machine and the two configurations shift by eleven. The final pair clause assembles the output configuration $\langle q',t'\rangle$. Each conjunction therefore corresponds to one component of the external *tm-step* definition.

lemma *step-fm-type* [*TC*]:
$[\![p \in nat;\ c \in nat;\ d \in nat]\!] \Longrightarrow$ *step-fm*(*p*,*c*,*d*) $\in$ *formula*
⟨*proof*⟩

context *M-ZF1-trans*
begin

lemma *configuration-in-A* [*simp*]: *c* $\in$ *configuration* $\Longrightarrow$ *c* $\in$ *M*
⟨*proof*⟩

lemma *sats-step-fm*:
assumes *idx*: *ip* $\in$ *nat ic* $\in$ *nat jd* $\in$ *nat* **and** *env*: *env* $\in$ *list*(*M*)
and *P*: *nth*(*ip*,*env*) = *P P* $\in$ *tm-machine*
and *c*: *nth*(*ic*,*env*) = *c c* $\in$ *configuration*
and *d*: *nth*(*jd*,*env*) = *d d* $\in$ *M*
shows (*M*, *env* $\models$ *step-fm*(*ip*,*ic*,*jd*)) $\longleftrightarrow$ *d* = *tm-step*(*P*,*c*)
⟨*proof*⟩

end

14.6 Run formulas

definition *init-fm* :: [*i*,*i*] $\Rightarrow$ *i* **where**
init-fm(*r*,*x*) $\equiv$
Exists(*Exists*(*Exists*(*Exists*(*Exists*(
And(*empty-fm*(*4*), *And*(*const-fm*(*1*,*3*), *And*(*Nil-fm*(*2*),
And(*pair-fm*(*2*,*x#+5*,*1*), *And*(*pair-fm*(*3*,*1*,*0*),
fun-apply-fm(*r#+5*,*4*,*0*)))))))))))

definition *trans-fm* :: [*i*,*i*,*i*] $\Rightarrow$ *i* **where**
trans-fm(*p*,*r*,*n*) $\equiv$
Forall(*Implies*(*Member*(*0*,*n#+1*),
Exists(*Exists*(*Exists*(
And(*succ-fm*(*3*,*2*), *And*(*fun-apply-fm*(*r#+4*,*3*,*1*),
And(*fun-apply-fm*(*r#+4*,*2*,*0*), *step-fm*(*p#+4*,*1*,*0*)))))))))

definition *final-fm* :: [*i*,*i*] $\Rightarrow$ *i* **where**
final-fm(*r*,*n*) $\equiv$
Exists(*Exists*(*Exists*(

And(*empty-fm*(*2*), *And*(*fun-apply-fm*(*r*#+*3*,*n*#+*3*,*1*), *pair-fm*(*2*,*0*,*1*))))))

definition *halt-fm* :: *i* **where**
halt-fm ≡
Exists(*And*(*nat-member-fm*(*0*),
Exists(*And*(*init-fm*(*0*,*3*), *And*(*trans-fm*(*2*,*0*,*1*), *final-fm*(*0*,*1*))))))

In *init-fm*, the five bound entries, in index order, are the initial configuration, its tape, the empty left list, state 1, and the time 0. In *trans-fm*, the bounded variable k and three existential witnesses give the environment prefix $[r`succ(k),r`k,succ(k),k]$. The call to *step-fm* relates indices 1 and 0 and shifts the machine index by four. In *final-fm*, the three witnesses are the final tape, the final configuration, and state 0, in index order.

The free environment of *halt-fm* is $[P,x]$. The two outer existential quantifiers bind a natural time and a run witness, producing $[r,n,P,x]$. Consequently the initial clause uses indices $(0,3)$, the transition clause uses $(2,0,1)$, and the final clause uses $(0,1)$. The run witness need only satisfy these finite application constraints. The external halting run supplies a finite function in the model, and induction identifies the constrained values of any internal witness with the external execution.

lemma *init-fm-type* [*TC*]: ⟦*r* ∈ *nat*; *x* ∈ *nat*⟧ ⟹ *init-fm*(*r*,*x*) ∈ *formula*
⟨*proof*⟩

lemma *trans-fm-type* [*TC*]:
⟦*p* ∈ *nat*; *r* ∈ *nat*; *n* ∈ *nat*⟧ ⟹ *trans-fm*(*p*,*r*,*n*) ∈ *formula*
⟨*proof*⟩

lemma *final-fm-type* [*TC*]: ⟦*r* ∈ *nat*; *n* ∈ *nat*⟧ ⟹ *final-fm*(*r*,*n*) ∈ *formula*
⟨*proof*⟩

lemma *halt-fm-type* [*TC*]: *halt-fm* ∈ *formula*
⟨*proof*⟩

context *M-ZF1-trans*
begin

lemma *ap-in-A* [*simp*]: *f* ∈ *M* ⟹ *a* ∈ *M* ⟹ *f*`*a* ∈ *M*
⟨*proof*⟩

lemma *sats-init-fm* [*simp*]:
⟦*ir* ∈ *nat*; *ix* ∈ *nat*; *env* ∈ *list*(*M*)⟧ ⟹
(*M*, *env* ⊨ *init-fm*(*ir*,*ix*)) ⟷ *nth*(*ir*,*env*)`*0* = *initial-config*(*nth*(*ix*,*env*))
⟨*proof*⟩

lemma *sats-final-fm* [*simp*]:
⟦*ir* ∈ *nat*; *jn* ∈ *nat*; *env* ∈ *list*(*M*)⟧ ⟹
(*M*, *env* ⊨ *final-fm*(*ir*,*jn*)) ⟷
(∃ *t*∈*M*. *nth*(*ir*,*env*)`*nth*(*jn*,*env*) = ⟨*0*,*t*⟩)

⟨*proof*⟩

lemma *sats-trans-fm* [*simp*]:
assumes $ip \in nat\ ir \in nat\ jn \in nat\ env \in list(M)$
$nth(jn,env) \in nat$
shows $(M,\ env \models trans\text{-}fm(ip,ir,jn)) \longleftrightarrow$
$(\forall\, k{\in}nth(jn,env).\ (M,\ Cons(nth(ir,env)`succ(k),$
$Cons(nth(ir,env)`k,$
$Cons(succ(k),\ Cons(k,\ env)))) \models step\text{-}fm(ip\#{+}4,1,0)))$
⟨*proof*⟩

14.7 Run correspondence

The following two lemmas separate finite execution from the layout of a surrounding formula. The reconstruction lemma accepts any initial configuration and any outer environment. Its induction establishes configuration validity along the run before applying the one-step theorem. The converse lemma supplies a transition from the canonical finite trace. These same interfaces support the halting proof and applications that constrain a run's output.

lemma *trans-run-steps*:
assumes *env*: $env \in list(M)$ **and** *p*: $p \in nat$
and *P*: $nth(p,env) = P\ P \in tm\text{-}machine$
and *c0*: $c0 \in configuration$ **and** *n*: $n \in nat$
and *rA*: $r \in M$ **and** *r0*: $r`0 = c0$
and *rs*: $\bigwedge k.\ k \in n \Longrightarrow$
$(M,\ Cons(r`succ(k),\ Cons(r`k,\ Cons(succ(k),\ Cons(k,\ env))))$
$\models step\text{-}fm(p\#{+}4,1,0))$
shows $r`n = steps(P,c0,n)$
⟨*proof*⟩

lemma *trace-trans*:
assumes *env*: $env \in list(M)$ **and** *p*: $p \in nat$
and *P*: $nth(p,env) = P\ P \in tm\text{-}machine$
and *c0*: $c0 \in configuration$ **and** *n*: $n \in nat$ **and** *k*: $k \in n$
shows $(M,\ Cons((\lambda j{\in}succ(n).\ steps(P,c0,j))`succ(k),$
$Cons((\lambda j{\in}succ(n).\ steps(P,c0,j))`k,$
$Cons(succ(k),\ Cons(k,\ env)))) \models step\text{-}fm(p\#{+}4,1,0))$
⟨*proof*⟩

14.8 Halting satisfaction

lemma *halting-witness-iff*:
assumes *P*: $P \in tm\text{-}machine$ **and** *x*: $x \in list(symbol)$
shows $(\exists\, n{\in}M.\ n \in nat \wedge (\exists\, r{\in}M.\ r`0 = initial\text{-}config(x) \wedge$
$(\forall\, k{\in}n.\ (M,\ Cons(r`succ(k),\ Cons(r`k,\ Cons(succ(k),\ Cons(k,$
$[r,n,P,x])))) \models step\text{-}fm(6,1,0))) \wedge (\exists\, t{\in}M.\ r`n = \langle 0,t\rangle))) \longleftrightarrow$
$halts\text{-}on(P,x)$

⟨*proof*⟩

theorem *sats-halt-fm*:
assumes *P*: $P \in$ *tm-machine* **and** *x*: $x \in$ *list*(*symbol*)
shows (*M*, [*P*,*x*] $\models$ *halt-fm*) $\longleftrightarrow$ *halts-on*(*P*,*x*)
⟨*proof*⟩

end

theorem *transitive-model-halt-fm-iff*:
assumes *transitive-zf-model*(*A*) $P \in$ *tm-machine* $x \in$ *list*(*symbol*)
shows (*A*, [*P*,*x*] $\models$ *halt-fm*) $\longleftrightarrow$ *halts-on*(*P*,*x*)
⟨*proof*⟩

14.9 Arities

context
notes *Un-assoc*[*symmetric*,*simp*] *Un-trasposition-aux1*[*simp*]
begin
⟨*ML*⟩
end

declare *arity-nat-name-fm*[*arity*]

context
notes *Un-assoc*[*symmetric*,*simp*] *Un-trasposition-aux1*[*simp*]
begin
⟨*ML*⟩
end

lemma *arity-halt-fm-eq*: *arity*(*halt-fm*) = *2*
⟨*proof*⟩
end

15 Closed Halting Sentences

theory *Turing-Halting-Sentences*
imports *Turing-Halting-Formula Turing-Tape*
begin

Closed sentences are obtained from *halt-fm* by naming the machine and the input with formulas that define them in every transitive model. The resulting satisfaction equation applies to any finite machine and symbol input in a transitive ZF set model.

15.1 Names for finite lists

lemma *nat-name-sym-type*: $a \in$ *symbol* $\Longrightarrow$ *nat-name-fm*(a) $\in$ *formula*
⟨*proof*⟩

lemma *arity-nat-name-sym*: $a \in symbol \Longrightarrow arity(nat\text{-}name\text{-}fm(a)) = 1$
⟨*proof*⟩

lemma *sats-nat-name-sym*:
$[\![M\text{-}ZF1\text{-}trans(A);\ a \in symbol;\ y \in A;\ env \in list(A)]\!] \Longrightarrow$
$(A,\ Cons(y,env) \models nat\text{-}name\text{-}fm(a)) \longleftrightarrow y = a$
⟨*proof*⟩

definition *list-name-fm* :: $[i \Rightarrow i,\ i] \Rightarrow i$ **where**
list-name-fm$(N,l) \equiv$
list-rec(*Nil-fm*(*0*), $\lambda a\ t\ p.$ *Exists*(*And*($N(a)$, *Exists*(*And*(p,
Cons-fm(*1*,*0*,*2*))))), l)

A list name has one free variable, at index 0. In the nonempty case it binds the head and then the tail. At *Cons-fm*(*1*,*0*,*2*), the environment prefix consists of the tail, the head, and the list being named. An instruction name uses the same binding pattern for its action and next-state fields, joined by *pair-fm*(*1*,*0*,*2*).

lemma *list-name-fm-Nil* [*simp*]: *list-name-fm*(N,[]) = *Nil-fm*(*0*)
⟨*proof*⟩

lemma *list-name-fm-Cons* [*simp*]:
list-name-fm(N,*Cons*(a,t)) = *Exists*(*And*($N(a)$, *Exists*(*And*(*list-name-fm*(N,t),
Cons-fm(*1*,*0*,*2*)))))
⟨*proof*⟩

lemma *list-name-fm-type* [*TC*]:
assumes l: $l \in list(B)$ **and** N: $\bigwedge a.\ a \in B \Longrightarrow N(a) \in formula$
shows *list-name-fm*(N,l) $\in$ *formula*
⟨*proof*⟩

lemma *arity-list-name-fm*:
assumes l: $l \in list(B)$ **and** N: $\bigwedge a.\ a \in B \Longrightarrow N(a) \in formula$
and aN: $\bigwedge a.\ a \in B \Longrightarrow arity(N(a)) = 1$
shows *arity*(*list-name-fm*(N,l)) = *1*
⟨*proof*⟩

lemma *sats-list-name-fm*:
assumes *model*: *M-ZF1-trans*(A) **and** l: $l \in list(B)$
and BA: $B \subseteq A$
and N: $\bigwedge a\ y\ env.\ a \in B \Longrightarrow y \in A \Longrightarrow env \in list(A) \Longrightarrow$
$(A,\ Cons(y,env) \models N(a)) \longleftrightarrow y = a$
shows $\bigwedge y\ env.\ y \in A \Longrightarrow env \in list(A) \Longrightarrow$
$(A,\ Cons(y,env) \models$ *list-name-fm*(N,l)$) \longleftrightarrow y = l$
⟨*proof*⟩

definition *instr-name-fm* :: $i \Rightarrow i$ **where**
instr-name-fm(i) $\equiv$

Exists(*And*(*nat-name-fm*(*fst*(*i*)), *Exists*(*And*(*nat-name-fm*(*snd*(*i*)),
pair-fm(*1*,*0*,*2*)))))

lemma *instr-name-fm-type* [*TC*]: $i \in \mathit{instruction} \Longrightarrow \mathit{instr\text{-}name\text{-}fm}(i) \in \mathit{formula}$
⟨*proof*⟩

lemma *arity-instr-name-fm*: $i \in \mathit{instruction} \Longrightarrow \mathit{arity}(\mathit{instr\text{-}name\text{-}fm}(i)) = 1$
⟨*proof*⟩

lemma *sats-instr-name-fm*:
assumes *model*: *M-ZF1-trans*(*A*) **and** *i*: $i \in \mathit{instruction}$
and *y*: $y \in A$ **and** *env*: $\mathit{env} \in \mathit{list}(A)$
shows $(A, \mathit{Cons}(y,\mathit{env}) \models \mathit{instr\text{-}name\text{-}fm}(i)) \longleftrightarrow y = i$
⟨*proof*⟩

15.2 Closed halting sentences

definition *halt-sentence* :: $[i,i] \Rightarrow i$ **where**
halt-sentence(*P*,*x*) ≡
Exists(*And*(*list-name-fm*(*nat-name-fm*,*x*),
Exists(*And*(*list-name-fm*(*instr-name-fm*,*P*), *halt-fm*))))

The outer quantifier names the input and the inner quantifier names the machine. The environment passed to *halt-fm* is therefore [*P*,*x*]. Both free variables of the halting formula are bound, and the resulting sentence has arity 0.

lemma *halt-sentence-type* [*TC*]:
assumes *P*: $P \in \mathit{tm\text{-}machine}$ **and** *x*: $x \in \mathit{list}(\mathit{symbol})$
shows $\mathit{halt\text{-}sentence}(P,x) \in \mathit{formula}$
⟨*proof*⟩

lemma *arity-halt-sentence*:
assumes *P*: $P \in \mathit{tm\text{-}machine}$ **and** *x*: $x \in \mathit{list}(\mathit{symbol})$
shows $\mathit{arity}(\mathit{halt\text{-}sentence}(P,x)) = 0$
⟨*proof*⟩

context *M-ZF1-trans*
begin

theorem *sats-halt-sentence-iff*:
assumes *P*: $P \in \mathit{tm\text{-}machine}$ **and** *x*: $x \in \mathit{list}(\mathit{symbol})$
shows $(M, [] \models \mathit{halt\text{-}sentence}(P,x)) \longleftrightarrow \mathit{halts\text{-}on}(P,x)$
⟨*proof*⟩

end

theorem *sats-halt-sentence-iff*:
assumes $\mathit{transitive\text{-}zf\text{-}model}(A)$ $P \in \mathit{tm\text{-}machine}$ $x \in \mathit{list}(\mathit{symbol})$
shows $(A, [] \models \mathit{halt\text{-}sentence}(P,x)) \longleftrightarrow \mathit{halts\text{-}on}(P,x)$

⟨*proof*⟩

The theorem *sats-halt-sentence-iff* explicitly assumes a transitive ZF set model. The preceding result in *M-ZF1-trans* uses weaker assumptions. Neither theorem asserts model existence.

15.3 Model-theoretic consequences

theorem *sats-halt-sentence-half-tape-eq-iff*:
  assumes *model*: *transitive-zf-model*(A) **and** P: $P \in$ *tm-machine*
    and x: $x \in$ *list*(*symbol*) **and** y: $y \in$ *list*(*symbol*)
    and *eq*: *half-tape-eq*(x,y)
  shows (A, [] $\models$ *halt-sentence*(P,x)) $\longleftrightarrow$
    (A, [] $\models$ *halt-sentence*(P,y))
  ⟨*proof*⟩

corollary *transitive-models-agree-on-halting*:
  assumes *transitive-zf-model*(A) *transitive-zf-model*(B)
    and $P \in$ *tm-machine* $x \in$ *list*(*symbol*)
  shows (A, [] $\models$ *halt-sentence*(P,x)) $\longleftrightarrow$ (B, [] $\models$ *halt-sentence*(P,x))
  ⟨*proof*⟩

end

16 Numerical Coding of Formulas

theory *Formula-Coding*
  imports
    Turing-Coding
    ZF−Constructible.Formula
begin

The primitive constructors are membership, equality, NAND, and universal quantification. Their tags are 0, 1, 2, and 3. Atomic formulas encode their two variable indices. NAND encodes the codes of its two subformulas, and universal quantification encodes the code of its body. Pairing makes the constructor tag and its arguments recoverable, so structural induction establishes injectivity.

16.1 Constructor codes

consts *formula-code* :: $i \Rightarrow i$

primrec
  formula-code(*Member*(x,y)) = *pair-code*(*0*,*pair-code*(x,y))
  formula-code(*Equal*(x,y)) = *pair-code*(*1*,*pair-code*(x,y))
  formula-code(*Nand*(p,q)) =
    pair-code(*2*,*pair-code*(*formula-code*(p),*formula-code*(q)))

formula-code(*Forall*(*p*)) = *pair-code*(*succ*(*2*),*formula-code*(*p*))

lemma *formula-code-type* [*TC*]:
p ∈ *formula* ⟹ *formula-code*(*p*) ∈ *nat*
⟨*proof*⟩

16.2 Injectivity

lemma *pair-code-inject*:
assumes *a*: *a* ∈ *nat* **and** *b*: *b* ∈ *nat*
and *c*: *c* ∈ *nat* **and** *d*: *d* ∈ *nat*
and *same*: *pair-code*(*a*,*b*) = *pair-code*(*c*,*d*)
shows *a* = *c* ∧ *b* = *d*
⟨*proof*⟩

lemma *pair-code-eq-iff*:
assumes *a* ∈ *nat b* ∈ *nat c* ∈ *nat d* ∈ *nat*
shows *pair-code*(*a*,*b*) = *pair-code*(*c*,*d*) ⟷ *a* = *c* ∧ *b* = *d*
⟨*proof*⟩

lemma *formula-code-injective*:
assumes *p* ∈ *formula q* ∈ *formula*
formula-code(*p*) = *formula-code*(*q*)
shows *p* = *q*
⟨*proof*⟩

end

17 Primitive-Recursive Formula Constructors

theory *Formula-Primrec*
imports *Formula-Coding Turing-Primrec*
begin

Primitive-recursive numerical functions implement the tagged formula constructors. Their application equations identify the output on formula codes with the code of the corresponding formula.

17.1 Tagged constructors

definition *pr-formula-binary* :: *i* ⇒ *i* **where**
pr-formula-binary(*k*) ≡
COMP(*pr-pair-code*,
[*CONSTANT*(*k*),*COMP*(*pr-pair-code*,[*PROJ*(*0*),*PROJ*(*1*)])])

lemma *pr-formula-binary-in-prim-rec* [*TC*]:
k ∈ *nat* ⟹ *pr-formula-binary*(*k*) ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-formula-binary-apply*:
assumes *k*: $k \in nat$ **and** *x*: $x \in nat$ **and** *y*: $y \in nat$
and *tail*: $l \in list(nat)$
shows *pr-formula-binary*(k)‘*Cons*$(x,Cons(y,l)) = pair\text{-}code(k,pair\text{-}code(x,y))$
⟨*proof*⟩

definition *pr-formula-unary* :: $i \Rightarrow i$ **where**
pr-formula-unary$(k) \equiv$
COMP(*pr-pair-code*,[*CONSTANT*(k),*PROJ*(0)])

lemma *pr-formula-unary-in-prim-rec* [*TC*]:
$k \in nat \Longrightarrow$ *pr-formula-unary*$(k) \in$ *prim-rec*
⟨*proof*⟩

lemma *pr-formula-unary-apply*:
assumes *k*: $k \in nat$ **and** *x*: $x \in nat$
and *tail*: $l \in list(nat)$
shows *pr-formula-unary*(k)‘*Cons*$(x,l) = pair\text{-}code(k,x)$
⟨*proof*⟩

17.2 Logical connectives

definition *pr-formula-neg* :: i **where**
pr-formula-neg $\equiv$
COMP(*pr-formula-binary*(2),[*PROJ*(0),*PROJ*(0)])

lemma *pr-formula-neg-in-prim-rec* [*TC*]:
pr-formula-neg $\in$ *prim-rec*
⟨*proof*⟩

lemma *pr-formula-neg-raw-apply*:
assumes *c*: $c \in nat$ **and** *tail*: $l \in list(nat)$
shows *pr-formula-neg*‘*Cons*$(c,l) = pair\text{-}code(2,pair\text{-}code(c,c))$
⟨*proof*⟩

lemma *pr-formula-neg-apply*:
assumes *p*: $p \in formula$ **and** *tail*: $l \in list(nat)$
shows *pr-formula-neg*‘*Cons*$(formula\text{-}code(p),l) = formula\text{-}code(Neg(p))$
⟨*proof*⟩

definition *pr-formula-and* :: i **where**
pr-formula-and $\equiv$
COMP(*pr-formula-neg*,
[*COMP*(*pr-formula-binary*(2),[*PROJ*(0),*PROJ*(1)])])

lemma *pr-formula-and-in-prim-rec* [*TC*]:
pr-formula-and $\in$ *prim-rec*
⟨*proof*⟩

lemma *pr-formula-and-apply*:
  assumes *p*: $p \in formula$ **and** *q*: $q \in formula$
    and *tail*: $l \in list(nat)$
  shows *pr-formula-and*`*Cons*(*formula-code*(*p*),*Cons*(*formula-code*(*q*),*l*)) =
    formula-code(*And*(*p*,*q*))
⟨*proof*⟩

definition *pr-formula-or* :: *i* **where**
  pr-formula-or ≡
    COMP(*pr-formula-binary*(*2*),
      [*COMP*(*pr-formula-neg*,[*PROJ*(*0*)]),
       COMP(*pr-formula-neg*,[*PROJ*(*1*)])])

lemma *pr-formula-or-in-prim-rec* [*TC*]:
  pr-formula-or ∈ *prim-rec*
  ⟨*proof*⟩

lemma *pr-formula-or-apply*:
  assumes *p*: $p \in formula$ **and** *q*: $q \in formula$
    and *tail*: $l \in list(nat)$
  shows *pr-formula-or*`*Cons*(*formula-code*(*p*),*Cons*(*formula-code*(*q*),*l*)) =
    formula-code(*Or*(*p*,*q*))
  ⟨*proof*⟩

definition *pr-formula-exists* :: *i* **where**
  pr-formula-exists ≡
    COMP(*pr-formula-neg*,
      [*COMP*(*pr-formula-unary*(*succ*(*2*)),
        [*COMP*(*pr-formula-neg*,[*PROJ*(*0*)])])])

lemma *pr-formula-exists-in-prim-rec* [*TC*]:
  pr-formula-exists ∈ *prim-rec*
  ⟨*proof*⟩

lemma *pr-formula-exists-apply*:
  assumes *p*: $p \in formula$ **and** *tail*: $l \in list(nat)$
  shows *pr-formula-exists*`*Cons*(*formula-code*(*p*),*l*) = *formula-code*(*Exists*(*p*))
⟨*proof*⟩

end

18 Primitive-Recursive Names for Numerals

theory *Turing-Name-Primrec*
  imports
    Formula-Primrec
    Turing-Halting-Sentences
begin

Iteration constructs the code of a formula naming a natural number. A second iteration names the unary input with that many one symbols. The application equations use the finite-object names whose satisfaction properties were established in *Turing-Halting-Sentences*.

18.1 Natural number names

definition *pr-nat-name-step* :: *i* **where**
pr-nat-name-step ≡
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*PROJ*(*0*),*CONSTANT*(*formula-code*(*succ-fm*(*0*,*1*)))])])

lemma *pr-nat-name-step-in-prim-rec* [*TC*]:
pr-nat-name-step ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-nat-name-step-apply*:
assumes *k*: *k* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-nat-name-step*‘*Cons*(*formula-code*(*nat-name-fm*(*k*)),*l*) =
formula-code(*nat-name-fm*(*succ*(*k*)))
⟨*proof*⟩

definition *pr-nat-name-code* :: *i* **where**
pr-nat-name-code ≡
PREC(*CONSTANT*(*formula-code*(*empty-fm*(*0*))),*pr-nat-name-step*)

lemma *pr-nat-name-code-in-prim-rec* [*TC*]:
pr-nat-name-code ∈ *prim-rec*
⟨*proof*⟩

theorem *pr-nat-name-code-apply*:
assumes *n*: *n* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-nat-name-code*‘*Cons*(*n*,*l*) = *formula-code*(*nat-name-fm*(*n*))
⟨*proof*⟩

18.2 Finite list names

definition *pr-list-name-cons* :: *i* **where**
pr-list-name-cons ≡
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*PROJ*(*0*),
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*PROJ*(*1*),*CONSTANT*(*formula-code*(*Cons-fm*(*1*,*0*,*2*)))])])])])

lemma *pr-list-name-cons-in-prim-rec* [*TC*]:
pr-list-name-cons ∈ *prim-rec*

⟨*proof*⟩

lemma *pr-list-name-cons-apply*:
assumes *p*: $p \in formula$ **and** *q*: $q \in formula$
and *tail*: $l \in list(nat)$
shows *pr-list-name-cons*‘*Cons*(*formula-code*(*p*),*Cons*(*formula-code*(*q*),*l*)) =
formula-code(*Exists*(*And*(*p*,*Exists*(*And*(*q*,*Cons-fm*(*1*,*0*,*2*))))))
⟨*proof*⟩

18.3 Unary input names

definition *pr-input-name-step* :: *i* **where**
pr-input-name-step ≡
COMP(*pr-list-name-cons*,
[*CONSTANT*(*formula-code*(*nat-name-fm*(*one-symbol*))),*PROJ*(*0*)])

lemma *pr-input-name-step-in-prim-rec* [*TC*]:
pr-input-name-step ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-input-name-step-apply*:
assumes *n*: $n \in nat$ **and** *tail*: $l \in list(nat)$
shows *pr-input-name-step*‘*Cons*(
formula-code(*list-name-fm*(*nat-name-fm*,*numeral-input*(*n*))),*l*) =
formula-code(*list-name-fm*(*nat-name-fm*,*numeral-input*(*succ*(*n*))))
⟨*proof*⟩

definition *pr-input-name-code* :: *i* **where**
pr-input-name-code ≡
PREC(*CONSTANT*(*formula-code*(*Nil-fm*(*0*))),*pr-input-name-step*)

lemma *pr-input-name-code-in-prim-rec* [*TC*]:
pr-input-name-code ∈ *prim-rec*
⟨*proof*⟩

theorem *pr-input-name-code-apply*:
assumes *n*: $n \in nat$ **and** *tail*: $l \in list(nat)$
shows *pr-input-name-code*‘*Cons*(*n*,*l*) =
formula-code(*list-name-fm*(*nat-name-fm*,*numeral-input*(*n*)))
⟨*proof*⟩

end

19 Primitive-Recursive Machine Names

theory *Turing-Machine-Name-Primrec*
imports *Turing-Name-Primrec*
Turing-Instruction-Normalization
begin

The machine decoder accepts every natural number. Instruction normalization supplies valid action and state fields before their names are constructed. The decoded instruction list has length at most its numeric code e. A primitive recursion of length e traverses the indices in reverse order and constructs the code of the complete list name.

19.1 Instruction names

definition *pr-instr-name-from-fields* :: i **where**
pr-instr-name-from-fields ≡
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*COMP*(*pr-nat-name-code*,[*PROJ*(*0*)]),
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*COMP*(*pr-nat-name-code*,[*PROJ*(*1*)]),
CONSTANT(*formula-code*(*pair-fm*(*1*,*0*,*2*)))])])])])

lemma *pr-instr-name-from-fields-in-prim-rec* [*TC*]:
pr-instr-name-from-fields ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-instr-name-from-fields-apply*:
assumes *a*: $a \in nat$ **and** *b*: $b \in nat$
and *tail*: $l \in list(nat)$
shows *pr-instr-name-from-fields*‘*Cons*(*a*,*Cons*(*b*,*l*)) =
formula-code(*instr-name-fm*(⟨*a*,*b*⟩))
⟨*proof*⟩

definition *pr-instr-name-code* :: i **where**
pr-instr-name-code ≡
COMP(*pr-instr-name-from-fields*,
[*COMP*(*pr-pair-left*,[*pr-normalize-instruction-code*]),
COMP(*pr-pair-right*,[*pr-normalize-instruction-code*])])

lemma *pr-instr-name-code-in-prim-rec* [*TC*]:
pr-instr-name-code ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-instr-name-code-apply*:
assumes *n*: $n \in nat$ **and** *tail*: $l \in list(nat)$
shows *pr-instr-name-code*‘*Cons*(*n*,*l*) =
formula-code(*instr-name-fm*(*decode-instruction*(*n*)))
⟨*proof*⟩

19.2 Decoded list indices

lemma *drop-at-index*:
assumes *k*: $k \in nat$ **and** *l*: $l \in list(A)$

shows *drop*(*k*,*l*) =
(*if* *k* < *length*(*l*) *then* *Cons*(*nth*(*k*,*l*),*drop*(*succ*(*k*),*l*)) *else* [])
⟨*proof*⟩

lemma *code-nth-decode*:
assumes *e*: *e* ∈ *nat* **and** *k*: *k* ∈ *nat*
shows *code-nth*(*k*,*e*) = *nth*(*k*,*nat-list-decode*(*e*))
⟨*proof*⟩

lemma *code-has-index-decode*:
assumes *e*: *e* ∈ *nat* **and** *k*: *k* ∈ *nat*
shows *code-has-index*(*k*,*e*) =
(*if* *k* < *length*(*nat-list-decode*(*e*)) *then* *1* *else* *0*)
⟨*proof*⟩

lemma *decode-machine-length*:
assumes *e*: *e* ∈ *nat*
shows *length*(*decode-machine*(*e*)) = *length*(*nat-list-decode*(*e*))
⟨*proof*⟩

lemma *decode-machine-nth*:
assumes *e*: *e* ∈ *nat* **and** *k*: *k* ∈ *nat*
and *index*: *k* < *length*(*decode-machine*(*e*))
shows *nth*(*k*,*decode-machine*(*e*)) = *decode-instruction*(*code-nth*(*k*,*e*))
⟨*proof*⟩

19.3 Bounded reverse traversal

At stage $k < e$, the selected index is $e \#- succ(k)$. The accumulator initially names the empty list. An absent index leaves it unchanged, and a present index prepends the name of the decoded instruction. After n stages, it names $drop(e \#- n, decode\text{-}machine(e))$. The decoded list has length at most e, so e stages construct its complete name without an unbounded search.

definition *pr-machine-reverse-index* :: *i* **where**
pr-machine-reverse-index ≡
COMP(*pr-diff*,[*PROJ*(*2*),*COMP*(*SC*,[*PROJ*(*1*)])])

lemma *pr-machine-reverse-index-in-prim-rec* [*TC*]:
pr-machine-reverse-index ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-machine-reverse-index-apply*:
assumes *acc*: *r* ∈ *nat* **and** *count*: *k* ∈ *nat*
and *tm-machine*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-reverse-index*‘*Cons*(*r*,*Cons*(*k*,*Cons*(*e*,*l*))) = *e* #− *succ*(*k*)
⟨*proof*⟩

definition *pr-machine-present* :: *i* **where**

pr-machine-present ≡
COMP(*pr-list-has-index*,[*pr-machine-reverse-index*,*PROJ*(*2*)])

lemma *pr-machine-present-in-prim-rec* [*TC*]:
pr-machine-present ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-machine-present-apply*:
assumes *acc*: *r* ∈ *nat* **and** *count*: *k* ∈ *nat*
and *tm-machine*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-present*‘*Cons*(*r*,*Cons*(*k*,*Cons*(*e*,*l*))) =
code-has-index(*e* #− *succ*(*k*),*e*)
⟨*proof*⟩

definition *pr-machine-element* :: *i* **where**
pr-machine-element ≡
COMP(*pr-list-nth*,[*pr-machine-reverse-index*,*PROJ*(*2*)])

lemma *pr-machine-element-in-prim-rec* [*TC*]:
pr-machine-element ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-machine-element-apply*:
assumes *acc*: *r* ∈ *nat* **and** *count*: *k* ∈ *nat*
and *tm-machine*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-element*‘*Cons*(*r*,*Cons*(*k*,*Cons*(*e*,*l*))) =
code-nth(*e* #− *succ*(*k*),*e*)
⟨*proof*⟩

definition *pr-machine-name-step* :: *i* **where**
pr-machine-name-step ≡
COMP(*pr-if-zero*,
[*pr-machine-present*,*PROJ*(*0*),
COMP(*pr-list-name-cons*,
[*COMP*(*pr-instr-name-code*,[*pr-machine-element*]),*PROJ*(*0*)])])

lemma *pr-machine-name-step-in-prim-rec* [*TC*]:
pr-machine-name-step ∈ *prim-rec*
⟨*proof*⟩

lemma *pr-machine-name-step-apply*:
assumes *q*: *q* ∈ *formula* **and** *k*: *k* ∈ *nat*
and *e*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-name-step*‘*Cons*(*formula-code*(*q*),*Cons*(*k*,*Cons*(*e*,*l*))) =
(*if code-has-index*(*e* #− *succ*(*k*),*e*) = *0 then formula-code*(*q*)
else formula-code(*Exists*(*And*(
instr-name-fm(*decode-instruction*(*code-nth*(*e* #− *succ*(*k*),*e*))),
Exists(*And*(*q*,*Cons-fm*(*1*,*0*,*2*)))))))
⟨*proof*⟩

definition *pr-machine-name-iter* :: *i* **where**
pr-machine-name-iter ≡
PREC(*CONSTANT*(*formula-code*(*Nil-fm*(*0*))),*pr-machine-name-step*)

lemma *pr-machine-name-iter-in-prim-rec* [*TC*]:
pr-machine-name-iter ∈ *prim-rec*
⟨*proof*⟩

theorem *pr-machine-name-iter-apply*:
assumes *e*: *e* ∈ *nat* **and** *n*: *n* ∈ *nat*
and *bound*: *n* ≤ *e* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-name-iter*‘*Cons*(*n*,*Cons*(*e*,*l*)) =
formula-code(*list-name-fm*(*instr-name-fm*,
drop(*e* #− *n*,*decode-machine*(*e*))))
⟨*proof*⟩

19.4 Complete machine names

definition *pr-machine-name-code* :: *i* **where**
pr-machine-name-code ≡
COMP(*pr-machine-name-iter*,[*PROJ*(*0*),*PROJ*(*0*)])

lemma *pr-machine-name-code-in-prim-rec* [*TC*]:
pr-machine-name-code ∈ *prim-rec*
⟨*proof*⟩

theorem *pr-machine-name-code-apply*:
assumes *e*: *e* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-name-code*‘*Cons*(*e*,*l*) =
formula-code(*list-name-fm*(*instr-name-fm*,*decode-machine*(*e*)))
⟨*proof*⟩

end

20 Effective Halting Sentences

theory *Turing-Halting-Codes*
imports *Turing-Machine-Name-Primrec*
begin

The machine code and unary input are independent arguments of a primitive-recursive sentence constructor. The resulting code determines a closed formula whose satisfaction is equivalent to the specified execution halting. The self-input constructor is the diagonal instance. Membership in *prim-rec* certifies numerical construction independently of a machine implementation of that function.

20.1 Closed halting sentence code

definition *pr-machine-input-sentence-code* :: *i* **where**
pr-machine-input-sentence-code ≡
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*COMP*(*pr-input-name-code*,[*PROJ*(*1*)]),
COMP(*pr-formula-exists*,
[*COMP*(*pr-formula-and*,
[*pr-machine-name-code*,*CONSTANT*(*formula-code*(*halt-fm*))])])])])

lemma *pr-machine-input-sentence-code-in-prim-rec* [*TC*]:
pr-machine-input-sentence-code ∈ *prim-rec*
⟨*proof*⟩

theorem *pr-machine-input-sentence-code-apply*:
assumes *e*: *e* ∈ *nat* **and** *n*: *n* ∈ *nat* **and** *tail*: *l* ∈ *list*(*nat*)
shows *pr-machine-input-sentence-code*‘*Cons*(*e*,*Cons*(*n*,*l*)) =
formula-code(*halt-sentence*(*decode-machine*(*e*),*numeral-input*(*n*)))
⟨*proof*⟩

The equation *pr-machine-input-sentence-code-apply* supplies a particular closed formula with the constructed code. The code interface below identifies any formula with that code. Injectivity removes the need for a separate formula decoder. Formula membership is a premise because the coding function is specified on formulas.

theorem *pr-machine-input-sentence-code-iff*:
assumes *e*: *e* ∈ *nat* **and** *n*: *n* ∈ *nat* **and** *p*: *p* ∈ *formula*
shows *formula-code*(*p*) = *pr-machine-input-sentence-code*‘[*e*,*n*] ⟷
p = *halt-sentence*(*decode-machine*(*e*),*numeral-input*(*n*))
⟨*proof*⟩

theorem *sats-machine-input-sentence-code-iff*:
assumes *model*: *transitive-zf-model*(*A*)
and *e*: *e* ∈ *nat* **and** *n*: *n* ∈ *nat* **and** *p*: *p* ∈ *formula*
and *code*: *formula-code*(*p*) = *pr-machine-input-sentence-code*‘[*e*,*n*]
shows (*A*, [] ⊨ *p*) ⟷
halts-on(*decode-machine*(*e*),*numeral-input*(*n*))
⟨*proof*⟩

20.2 Self-input instance

definition *pr-halt-sentence-code* :: *i* **where**
pr-halt-sentence-code ≡
COMP(*pr-machine-input-sentence-code*,[*PROJ*(*0*),*PROJ*(*0*)])

lemma *pr-halt-sentence-code-in-prim-rec* [*TC*]:
pr-halt-sentence-code ∈ *prim-rec*
⟨*proof*⟩

theorem *pr-halt-sentence-code-apply*:
assumes *e*: $e \in nat$ **and** *tail*: $l \in list(nat)$
shows *pr-halt-sentence-code*‘$Cons(e,l) =$
$formula\text{-}code(halt\text{-}sentence(decode\text{-}machine(e),numeral\text{-}input(e)))$
⟨*proof*⟩

corollary *transitive-zf-contains-decoded-machine*:
$[\![transitive\text{-}zf\text{-}model(A);\ e \in nat]\!] \Longrightarrow decode\text{-}machine(e) \in A$
⟨*proof*⟩

end

21 Using the Halting Formula Interface

theory *Turing-Halting-Examples*
imports *Turing-Examples Turing-Halting-Codes*
begin

The operational proof from the machine example supplies an internal theorem in every transitive ZF set model. The same result applies to a formula identified only by its numerical code. Both clients use the public interfaces without unfolding the halting formula, its finite-object names, or the primitive-recursive constructor.

21.1 From execution to satisfaction

theorem *scan-append-sentence*:
assumes *model*: *transitive-zf-model*(A) **and** *n*: $n \in nat$
shows $(A, [] \models halt\text{-}sentence(scan\text{-}append\text{-}machine,numeral\text{-}input(n)))$
⟨*proof*⟩

21.2 From a sentence code to satisfaction

theorem *scan-append-sentence-code*:
assumes *model*: *transitive-zf-model*(A) **and** *n*: $n \in nat$
and *p*: $p \in formula$
and *code*: $formula\text{-}code(p) =$
$pr\text{-}machine\text{-}input\text{-}sentence\text{-}code‘[encode\text{-}machine(scan\text{-}append\text{-}machine),n]$
shows $(A, [] \models p)$
⟨*proof*⟩

end

References

[1] Jian Xu, Xingyuan Zhang, and Christian Urban. *Mechanising Turing Machines and Computability Theory in Isabelle/HOL*. In *Interactive Theorem Proving*, LNCS 7998, 147–162. Springer, 2013. https://doi.org/10.1007/978-3-642-39634-2_13.

[2] Emmanuel Gunther, Miguel Pagano, Pedro Sánchez Terraf, and Matías Steinberg. *The Independence of the Continuum Hypothesis in Isabelle/ZF*. Archive of Formal Proofs, 2022. https://isa-afp.org/entries/Independence_CH.html.

[3] Jian Xu, Xingyuan Zhang, Christian Urban, Sebastiaan J. C. Joosten, and Franz Regensburger. *Universal Turing Machine*. Archive of Formal Proofs, 2019, revised 2022. https://isa-afp.org/entries/Universal_Turing_Machine.html.

[4] Emmanuel Gunther, Miguel Pagano, Pedro Sánchez Terraf, and Matías Steinberg. *Transitive Models of Fragments of ZFC*. Archive of Formal Proofs, 2022. https://isa-afp.org/entries/Transitive_Models.html.

[5] Thomas Jech. *Set Theory: The Third Millennium Edition, Revised and Expanded*. Springer, 2003. https://doi.org/10.1007/3-540-44761-X.

[6] Lawrence C. Paulson. *The Relative Consistency of the Axiom of Choice, Mechanized Using Isabelle/ZF*. LMS Journal of Computation and Mathematics, 6, 198–248, 2003.

[7] Nora Szasz. *A Machine Checked Proof that Ackermann's Function is not Primitive Recursive*. In Gérard Huet and Gordon Plotkin, editors, *Logical Environments*, 317–338. Cambridge University Press, 1993.